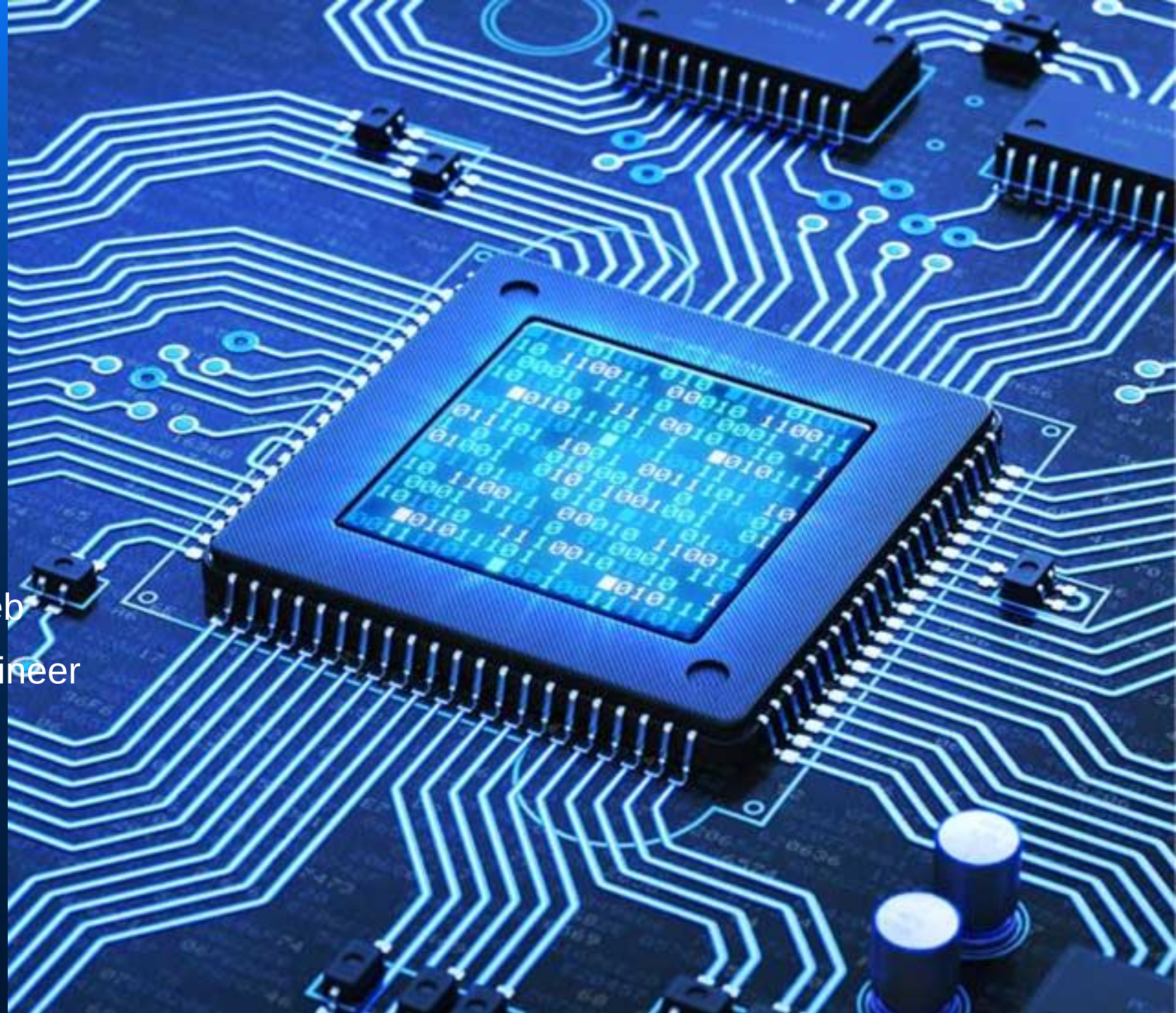
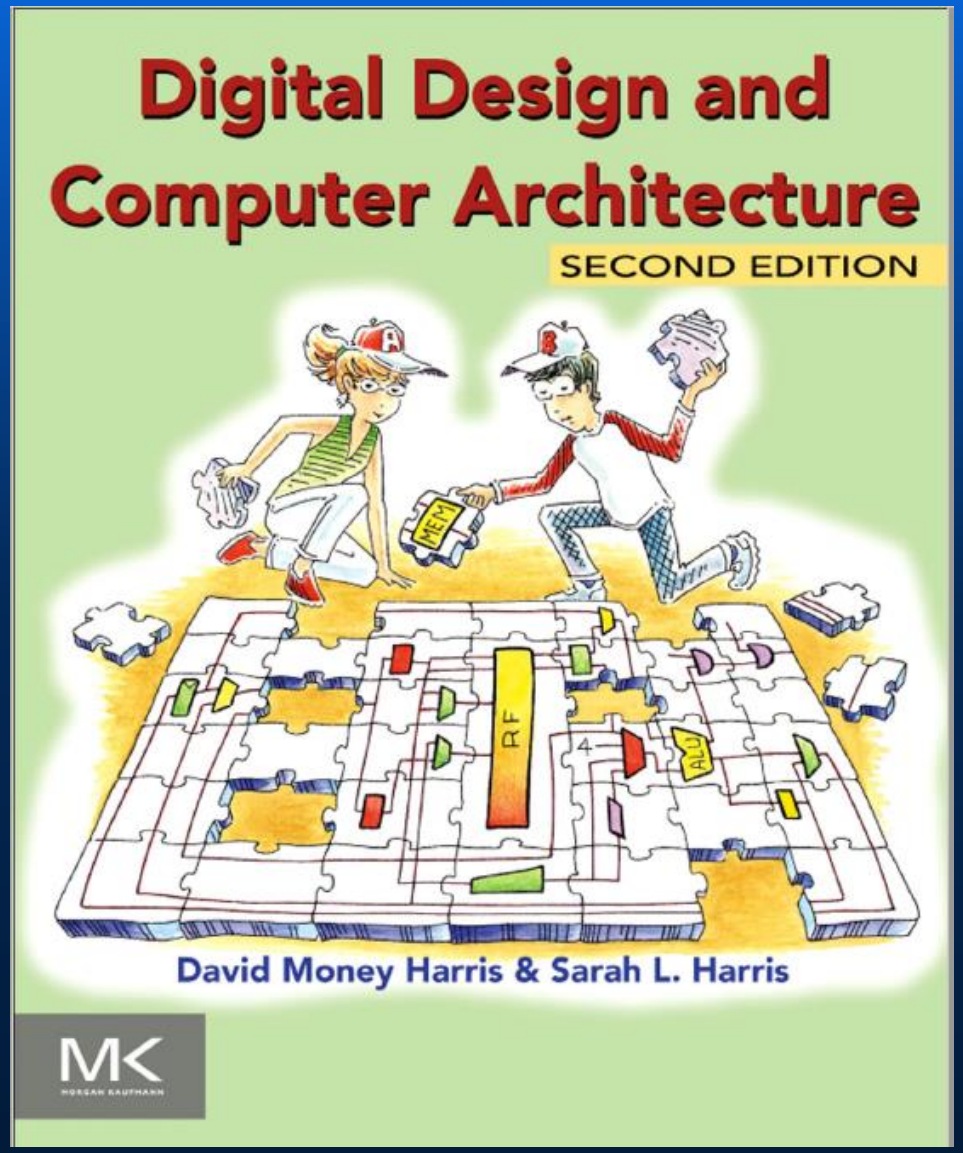
Computer Architecture Session 3

By: Eng. Abdulkareem Abotaleb

Senior HW Digital Design Engineer





This presentation is prepared from:

Digital Design and Computer Architecture, 2nd Edition.
David Harris and Sarah L. Harris.

Agenda

Chapter 6: Architecture

- # 7.1 Exceptions
- # 7.2 Explanation of Some Instructions

Chapter 7: Microarchitecture

- # 4. Multi-Cycle Processor

7. Exceptions

life.augmented

7.2 Exceptions

- An exception is like an unscheduled function call that jumps to a new address.
- Exceptions may be caused by hardware or software.
- **Hardware Exception (interrupt):**
  - It's triggered by an input/ output (I/O) device like a keyboard.
  - The processor may receive notification that a key is pressed. The processor may stop what it is doing, determine which key was pressed, save it for future reference, then resume the program that was running.
- **Software Exception (traps):**
  - The program may encounter an error condition such as an undefined instruction.
  - The program jumps to code in the OS, which may choose to terminate the program.
  - Other causes of exceptions include division by zero, attempts to read nonexistent memory, hardware malfunctions, debugger breakpoints, and arithmetic overflow.

7.2 Exceptions

What we need to implement in hardware:

- The processor records the cause of an exception and the PC value when the exception occurs.
- Then, it jumps to the exception handler function.
- The exception handler is code (in the OS) that reads the cause of the exception and responds appropriately (by reading the keyboard on a hardware interrupt, for example). It then returns to the program that was executing before the exception took place.
- In MIPS, the exception handler is always located at 0x80000180.
- When an exception occurs, the processor always jumps to this address, regardless of the cause.
- MIPS architecture uses a special-purpose register **(Cause)** to determine how to handle the exception.
- Some other architectures jump to a different exception handler for each different cause.

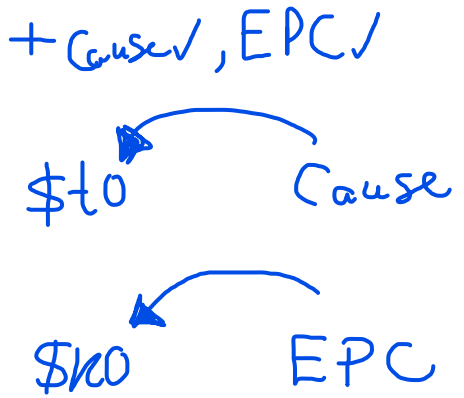
Exception	Cause
hardware interrupt	0x00000000
system call	0x00000020
breakpoint/divide by 0	0x00000024
undefined instruction	0x00000028
arithmetic overflow	0x00000030

7.2 Exceptions

- MIPS uses another special-purpose register called **the Exception Program Counter (EPC)** to store the value of the PC when the exception takes place. [like using $ra with **jal** instruction].
- The **EPC** and **Cause** registers are not part of the MIPS register file.

they are located in co-processor unit

- Example:
  - When an exception causes, the processor jumps to the exception handler.
  - The exception handler saves registers on the stack.
  - The handler copies the **Cause** register into $t0, using **mfc0** $t0, Cause (mfc0: **m**ove **f**rom **c**oprocessor **0** instruction).
  - When the handler finishes, it restores the registers from the stack.
  - Then, it copies the return address from **EPC** to $k0 using **mfc0** $k0, EPC.
  - Finally, it returns using **jr** $k0.



mfc0 rt, rd
[rt] = [rd]

31–26	25–21	20–16	15–11	10–0
Opcode 01000	rs(MF) 00000	rt	rd	00000000000

mtc0 rt, rd
[rd] = [rt]

31–26	25–21	20–16	15–11	10–0
Opcode 01000	rs(MT) 00100	rt	rd	00000000000

8. Explanation of Some Instructions

Agenda

8. Explanation of Some Instructions

8.1 Pseudoinstructions

8.2 Signed and Unsigned Instructions

8.3 Floating-Point Instructions

8.1 Pseudoinstructions

- MIPS defines **pseudoinstructions** that aren't part of the instruction set but are commonly used.
- Pseudoinstructions are translated into one or more MIPS instructions.
- Example: the load immediate (**li**) loads a 32-bit constant using the **lui** and **ori** instructions.
- The no operation pseudoinstruction (**nop**) performs no operation. Only, PC is incremented by 4.
- The machine code for the nop instruction is 0x00000000.

Pseudoinstruction	Corresponding MIPS Instructions
li $s0, 0x1234AA77	lui $s0, 0x1234 ori $s0, 0xAA77
clear $t0	add $t0, $0, $0
move $s2, $s1	add $s2, $s1, $0
nop	sll $0, $0, 0

8.1 Pseudoinstructions

- Some pseudoinstructions require a temporary register for intermediate calculations.
- Example: the pseudoinstruction **beq** compares $t2 to a 16-bit immediate (imm15:0).
  - It requires a temporary register in which to store the 16-bit immediate.
  - Assemblers use the assembler register, $at, for such purposes.

Pseudoinstruction	Corresponding MIPS Instructions
beq $t2, imm15:0, Loop	addi $at, $0, imm15:0 beq $t2, $at, Loop

Agenda

- # 8. Explanation of Some Instructions
  - # 8.1 Pseudoinstructions
  - # 8.2 Signed and Unsigned Instructions
  - # 8.3 Floating-Point Instructions

8.2 Signed and Unsigned Instructions

- The binary numbers may be signed or unsigned.
- MIPS architecture uses two’s complement representation of signed numbers.
- MIPS has certain instructions that may come in signed and unsigned:
  - Addition and Subtraction.
  - Multiplication and Division.
  - Set less than.
  - Partial word loads.

8.2 Signed and Unsigned Instructions

Addition and Subtraction

- Addition and subtraction are performed identically if the number is signed or unsigned. However, the results is different.
- Adding two large signed numbers may produce the opposite sign.
  Example: adding 0x7FFFFFFF + 0x7FFFFFFF = 0xFFFFFFFE = −2 (arithmetic overflow).
  Similarly, adding 0x80000001 + 0x80000001 = 0x00000002 (arithmetic overflow).
- MIPS processor takes an exception on arithmetic overflow.
- The program can decide what to do. It might repeat the calculation with greater precision to avoid the overflow, then return to where it stopped.
- The signed instr are **add**, **addi**, and **sub**, while the unsigned are **addu**, **addiu**, and **subu**.
- They both are identical except that signed versions trigger an exception on overflow, whereas unsigned instructions do not.

if addu: no arithmetic overflow

8.2 Signed and Unsigned Instructions

Addition and Subtraction

Example(1):

	unsigned	signed
•	1011 => +11	(-0101) -5
•	1100 => +12	(-0100) -4
•	==========	
• With Carry:	1 0111 => 23	(-0 1001) -9
• MIPS:	0111 => 7 (Wrong)	+7 (Wrong)
	No_Overflow	Overflow

8.2 Signed and Unsigned Instructions

Addition and Subtraction

Example(2):

	unsigned	signed
•	1011 => +11	(-0101) -5
•	0100 => +4	(0100) +4
•	==========	==========
• With Carry:	0 1111 => 16	(- 0001) -1
• MIPS:	1111 => 16 (Correct)	-1 (Correct)
	No_Overflow	No_Overflow

8.2 Signed and Unsigned Instructions

Addition and Subtraction

Example(3):

	unsigned	signed
•	0011 => +3	+3
•	0100 => +4	+4
•	==========	
• With Carry:	0111 => 7 (Correct)	+7 (Correct)
• MIPS:	0111 => 7 (Correct)	+7 (Correct)
	No_Overflow	No_Overflow

addu never produces an overflow

8.2 Signed and Unsigned Instructions

unlike Add/SUB

Multiplication and Division

- Multiplication and division behave differently for signed and unsigned numbers.
- Example: as an unsigned number, 0xFFFFFFFF represents a large number, but as a signed number it represents –1.
  Hence, 0xFFFFFFFF × 0xFFFFFFFF would equal 0xFFFFFFFE00000001 if the numbers were unsigned but 0x0000000000000001 if the numbers were signed.
- Therefore, multiplication and division come in both signed and unsigned flavors.
- **mult** and **div** treat the operands as signed numbers.
- **multu** and **divu** treat the operands as unsigned numbers.



A result depending on your representation

In signed case, we should multiply the absolutes and put the sign at the end as an algorithm

8.2 Signed and Unsigned Instructions

Set Less Than

- It compares either two registers (**slt**) or a register and an immediate (**slti**).
- Set less than also comes in signed (**slt** and **slti**) and unsigned (**sltu** and **sltiu**) versions.
- Example:
  - In a signed comparison, 0x80000000 is less than any other number, because it is the most negative two's complement number.
  - In an unsigned comparison, 0x80000000 is greater than 0x7FFFFFFF but less than 0x80000001, because all numbers are positive.
- Beware that **slti-sltiu** sign-extends the immediate before operating.
  Example: **sltiu** $s0, $s1, 0x8042 compares $s1 to 0xFFFF8042 (a large +ve number).

?!

?

wierd choice

8.2 Signed and Unsigned Instructions

Loads

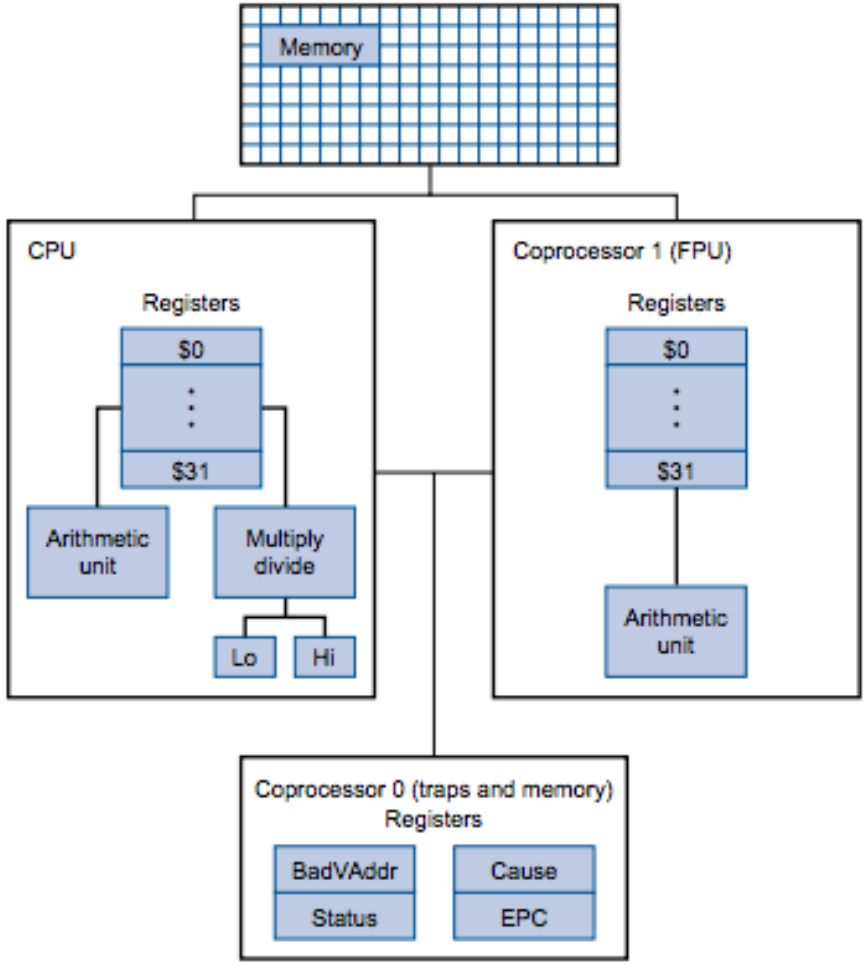
- Byte loads come in signed (**lb**) and unsigned (**lbu**) versions.
- **lb-lh** sign-extends the entire 32-bit register,
  which load the lower half and sign-extend the upper two/three bytes of the word.
- **lbu-lhu** zero-extends the entire 32-bit register,
  which load the lower half and zero-extend the upper two/three bytes of the word.

Agenda

- # 8. Explanation of Some Instructions
  - # 8.1 Pseudoinstructions
  - # 8.2 Signed and Unsigned Instructions
  - # 8.3 Floating-Point Instructions

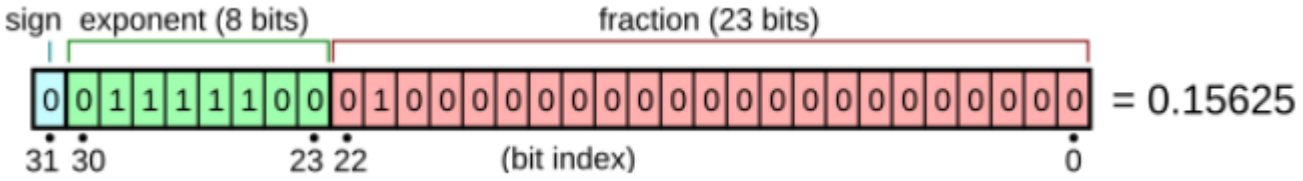
8.3 Floating-Point Instructions

- MIPS architecture defines an optional floating-point coprocessor (coprocessor 1).
- It was a separate chip that users could purchase if needed, but now it is built in the main processor in most recent MIPS implementations.
- MIPS defines 32 floating-point registers (32-bit), $f0–$f31.

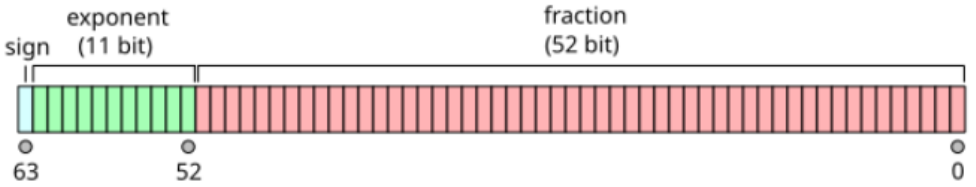


8.3 Floating-Point Instructions

- MIPS supports both single- and double-precision IEEE floating-point arithmetic.



$$(-1)^{b_{31}} \times 2^{(b_{30}b_{29}\ldots b_{23})_2 - 127} \times (1.b_{22}b_{21}\ldots b_0)_2,$$



$$(-1)^{\text{sign}}(1.b_{51}b_{50}\ldots b_0)_2 \times 2^{e-1023}$$

Example:

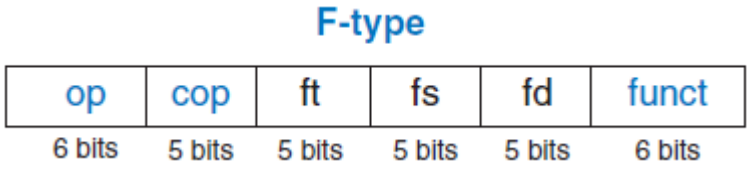
- Sign: $(-1)^{\text{sign}} = (-1)^0 = +1$
- Exponent: $E = (b_{30}b_{29}\ldots b_{23})_2 = \sum_{i=0}^{7} b_{23+i}2^{+i} = 124$ $\quad 2^{(E-127)} = 2^{124-127} = 2^{-3}$
- Fraction: $1.b_{22}b_{21}\ldots b_0 = 1 + \sum_{i=1}^{23} b_{23-i}2^{-i} = 1 + 1 \cdot 2^{-2} = 1.25$

- $\text{value} = (+1) \times 2^{-3} \times 1.25 = +0.15625.$

8.3 Floating-Point Instructions

- MIPS supports both single- and double-precision IEEE floating-point arithmetic.
- Double-precision (64-bit) numbers are stored in pairs of 32-bit registers, so only the 16 even-numbered regs ($f0, $f2, $f4, . . . , $f30) are used to specify double-precision.
- Table has certain registers are reserved for certain purposes.

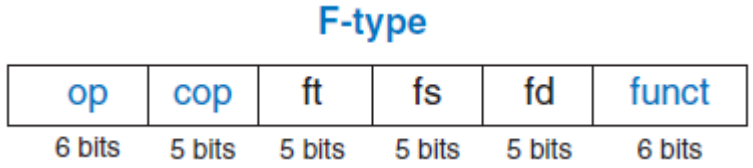
Name	Number	Use
$fv0–$fv1	0, 2	function return value
$ft0–$ft3	4, 6, 8, 10	temporary variables
$fa0–$fa1	12, 14	function arguments
$ft4–$ft5	16, 18	temporary variables
$fs0–$fs5	20, 22, 24, 26, 28, 30	saved variables



- MIPS defines the F-type instruction format for floating-point instructions.
- Floating-point instructions all have an opcode of 17 (10001_2).

8.3 Floating-Point Instructions

- Floating-point instructions require both a funct and a cop (coprocessor) field to indicate the type of instruction.



- Floating-point instructions come in both single- and double-precision flavors.
- cop = 16 (10000_2) for single-precision instr or 17 (10001_2) for double-precision instr.
- Like R-type instr, F-type instr have two source operands (fs and ft) and one dest (fd).
- Instruction precision is indicated by .s and .d in the mnemonic.
- Floating-point arithmetic instructions include addition (**add.s**, **add.d**), subtraction (**sub.s**, **sub.d**), multiplication (**mul.s**, **mul.d**), and division (**div.s**, **div.d**) as well as negation (**neg.s**, **neg.d**) and absolute value (**abs.s**, **abs.d**).

8.3 Floating-Point Instructions

- Floating-point branches have two parts.
- First, a compare instr is used to set or clear the floating-point condition flag (fpcond). Then, a conditional branch checks the value of the flag.
- The compare instructions include equality (**c.seq.s**/**c.seq.d**), less than (**c.lt.s**/**c.lt.d**), and less than or equal to (**c.le.s**/**c.le.d**).
- The conditional branch instr are (**bc1f** / **bc1t**) that branch if fpcond is FALSE/TRUE.
- Inequality, greater than or equal to, and greater than comparisons are performed with **seq**, **lt**, and **le**, followed by **bc1f**.
- Floating-point registers are loaded and stored from memory using **lwc1** and **swc1**.
- These instr move 32 bits, so two are necessary to handle a double-precision number.

4. Multi-Cycle Processor

4. Multi-Cycle Processor

- The weakness of single-cycle processor:
  1. It requires a clock cycle long enough to support the slowest instruction (lw).
  2. It requires three adders (one in the ALU and two for the PC logic).
  3. It has separate instruction and data memories, which may not be realistic.
- The multicycle processor divides the instruction into multiple shorter steps, so simpler instructions can complete faster than more complex ones.
- The processor needs only one adder which is reused for different purposes on steps.
- The processor uses a combined memory for instructions and data. The instruction is fetched from memory on the first step, and data may be read or written on later steps.

4. Multi-Cycle Processor

Following the same procedure, we design a multicycle processor.

- Construct a datapath by connecting the architectural state elements and memories with combinational logic.
- Add nonarchitectural state elements to hold intermediate results between the steps.
- Design the controller which produces different signals on different steps during execution of a single instruction, (a finite state machine rather than combinational logic).
- Examine how to add new instructions to the processor.
- Analyze the performance and compare to the single-cycle processor.

4. Multi-Cycle Processor

4.1 Multi-Cycle Datapath

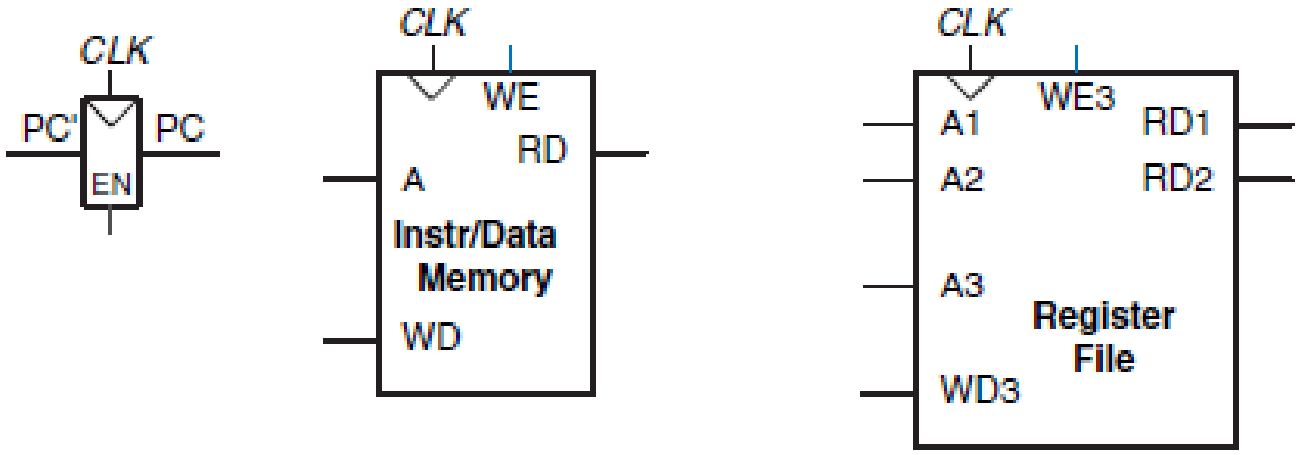
4.2 Multi-Cycle Control

4.3 More Instructions

4.4 Performance Analysis

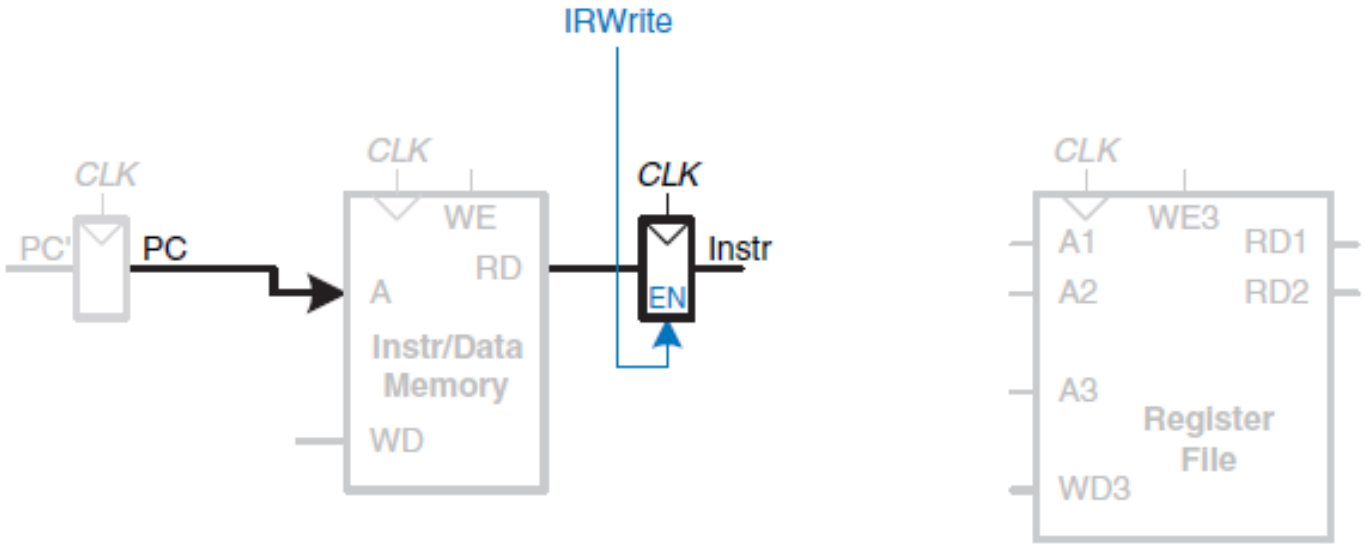
4. Multi-Cycle Processor

- The PC and register file consist the architectural state of the MIPS processor.
- A combined memory is used for both instructions and data which is more realistic.
- The PC contains the address of the instruction to execute.



4. Multi-Cycle Processor

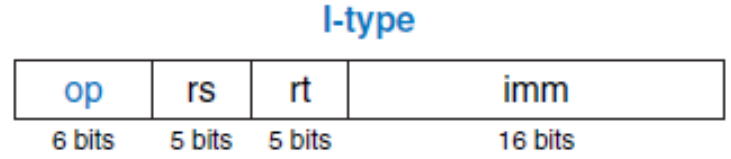
- The first step is to read this instruction from instruction memory.
- The PC is simply connected to the address input of the instruction memory. The instruction is read and stored in a new nonarchitectural Instruction Register.
- The Instruction Register receives an enable signal, IRWrite, which is set when it should be updated with a new instruction.

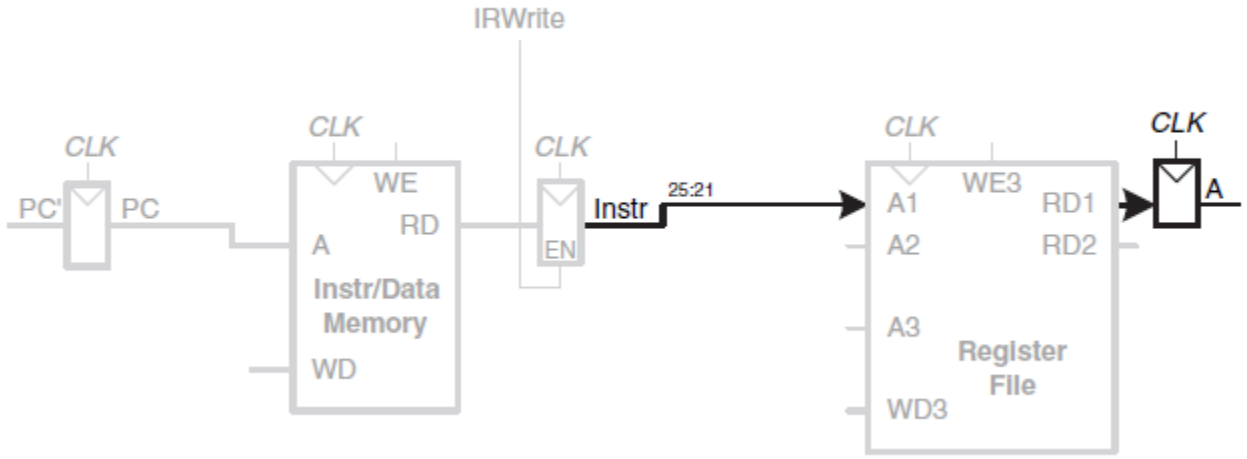


4. Multi-Cycle Processor

To handle **lw** instruction:

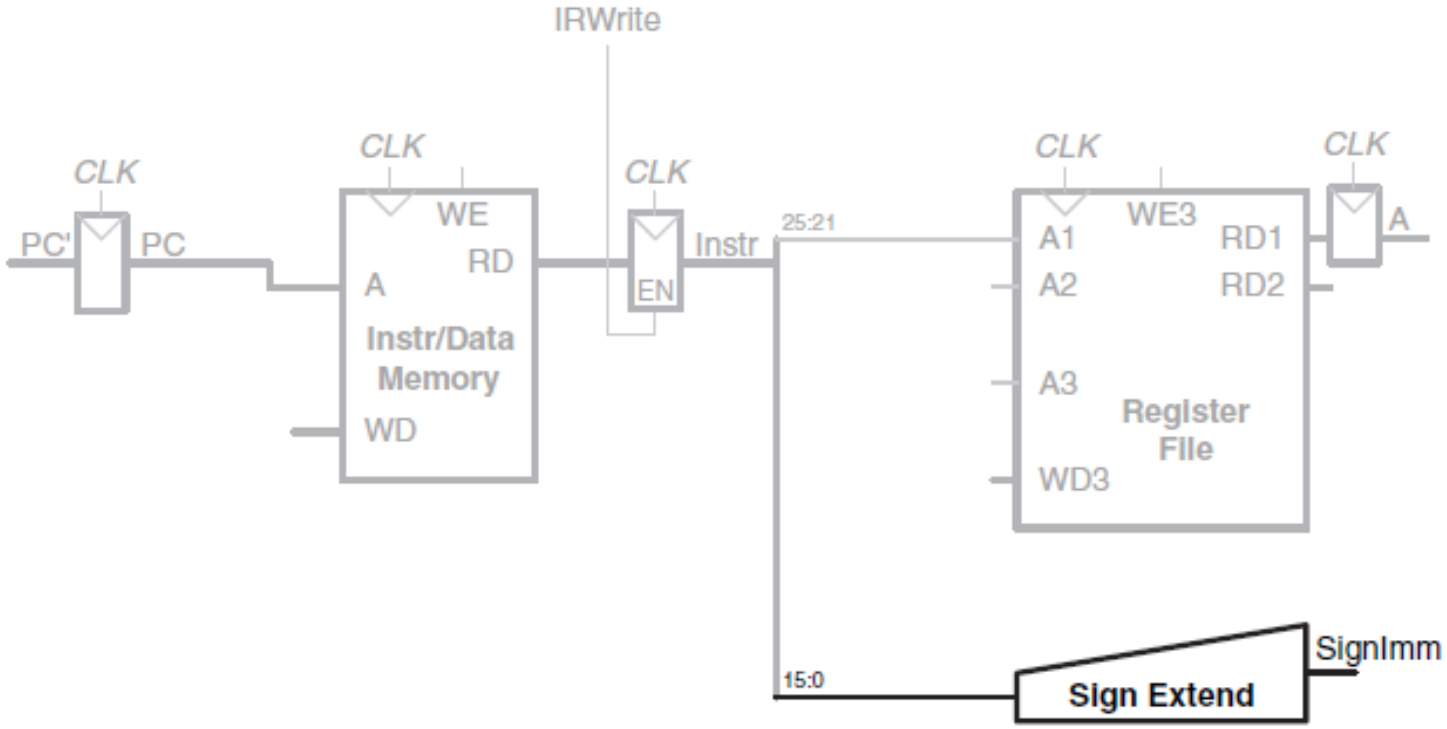
- The source register rs, Instr [25:21], is read containing the base address.
- Instr [25:21] is connected to one of the address inputs, A1, of the register file.
- The register file reads the register onto RD1.
- This value is stored in another nonarchitectural register, A.





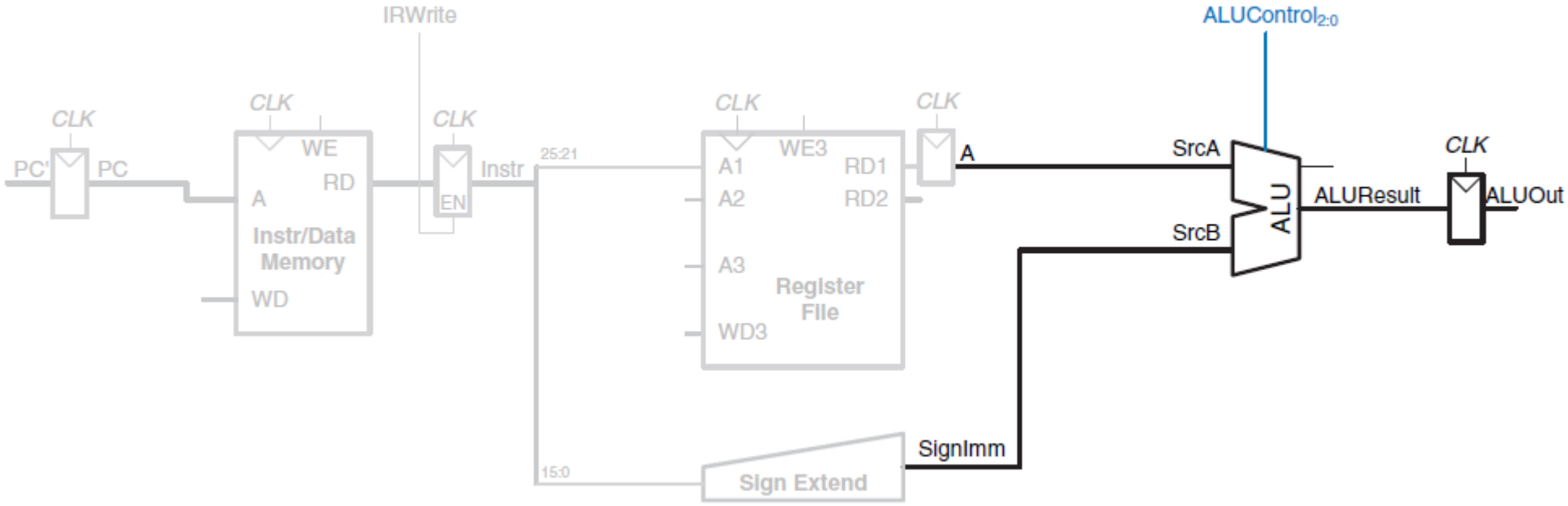
4. Multi-Cycle Processor

- The **lw** instruction reads the offset (the immediate field), Instr [15:0], and must be sign-extended to 32 bits (SignImm).
  No need to dedicate a register to hold SignImm as it's a combinational function of Instr and will not change while the current instruction is being processed.



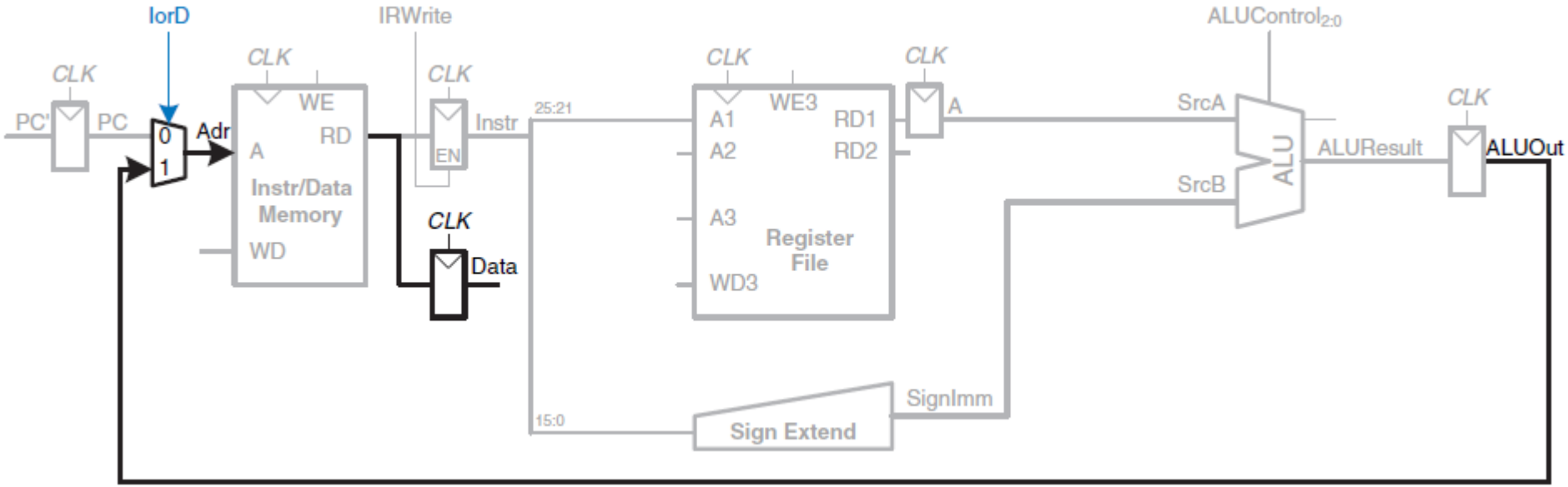
4. Multi-Cycle Processor

- The address of the load is the sum of the base address and offset.
- We use an ALU to compute this sum.
- ALUControl should be set to 010 to perform an addition.
- ALUResult is stored in a nonarchitectural register called ALUOut.



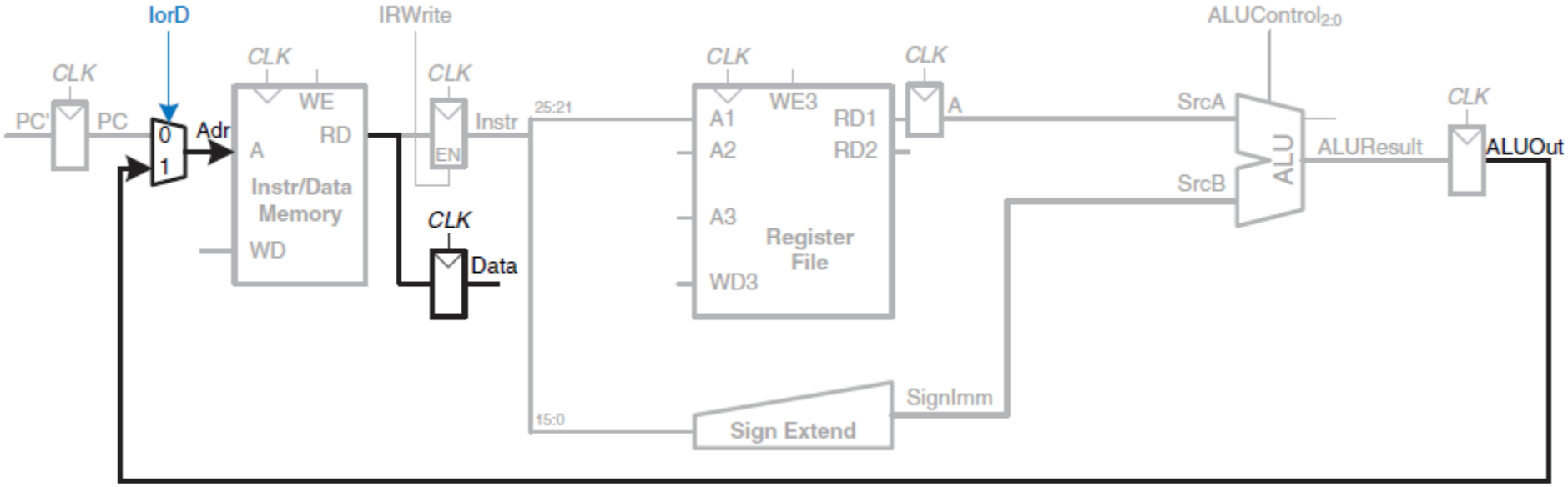
4. Multi-Cycle Processor

- We add a multiplexer in front of the memory to reuse the memory by choosing the address, Adr, from either the PC or ALUOut (data from the calculated address).
- The mux select signal, IorD, indicates either an instruction or data address.
- The data read from the memory is stored in another nonarchitectural register, called Data.



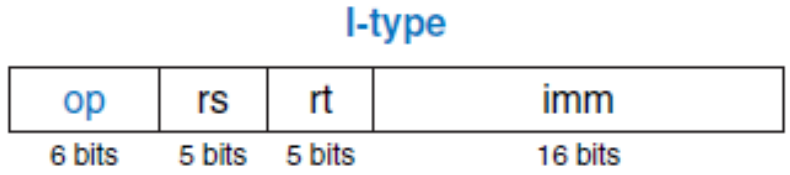
4. Multi-Cycle Processor

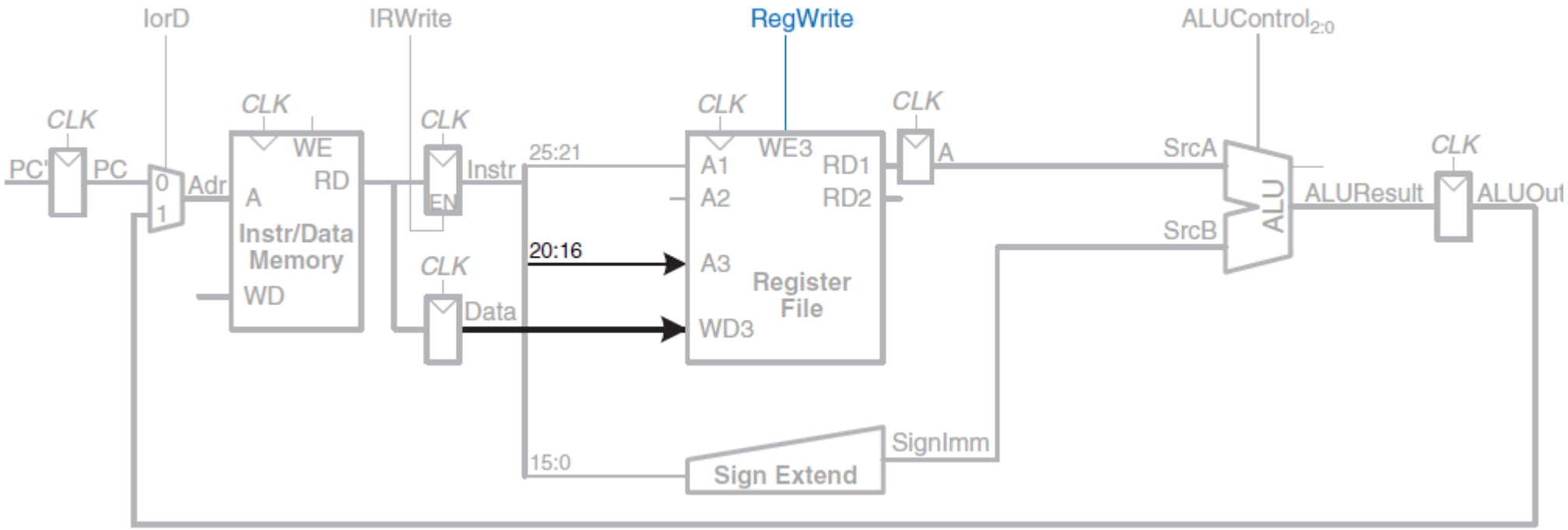
- On the first step, the address is taken from the PC to fetch the instruction.
- On a later step, the address is taken from ALUOut to load the data.
- Hence, IorD must have different values on different steps.
- The implemented FSM controller should generate these sequences of control signals.



4. Multi-Cycle Processor

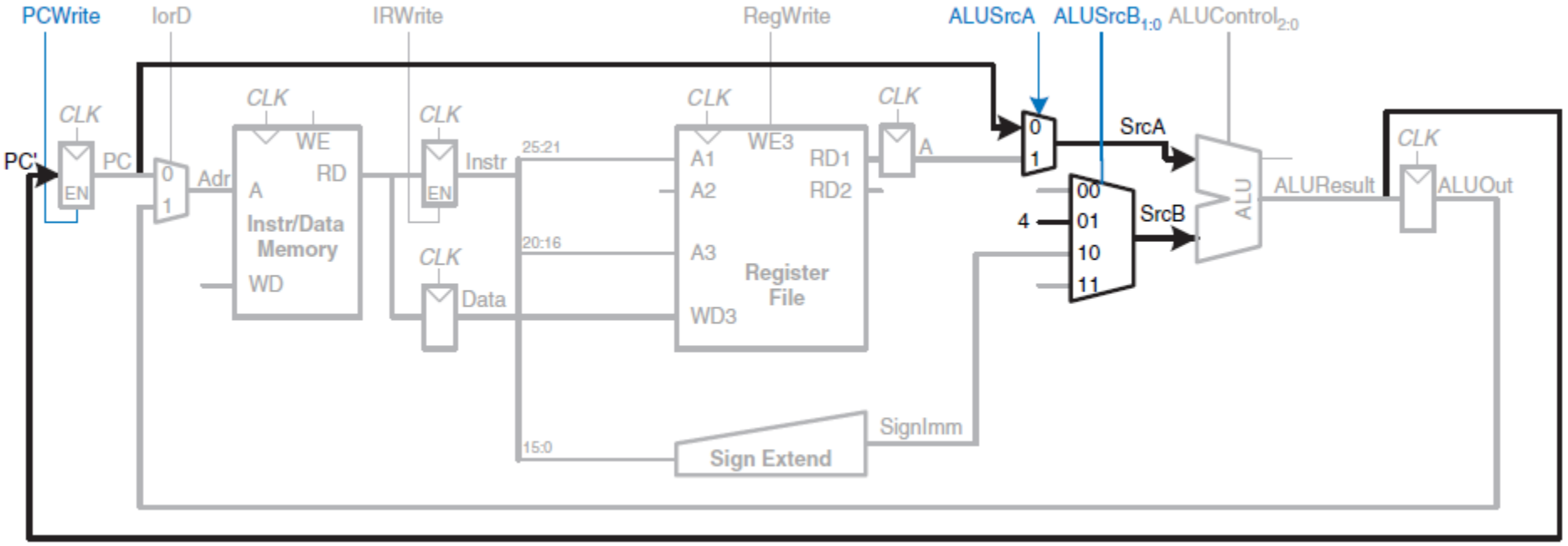
- Finally, the data is written back to the destination register specified by the rt field of the instruction, Instr [20:16].





4. Multi-Cycle Processor

- The existing ALU is used to update the program counter by adding 4 to the old PC.
- To choose the PC and the constant 4 as ALU inputs.
  - Insert 2-input multiplexer controlled by ALUSrcA to select either the PC or A reg as SrcA.
  - Insert 4-input multiplexer controlled by ALUSrcB to select either 4 or Sign-Imm as SrcB.
- The PCWrite control signal enables the PC register to be written only on certain cycles.



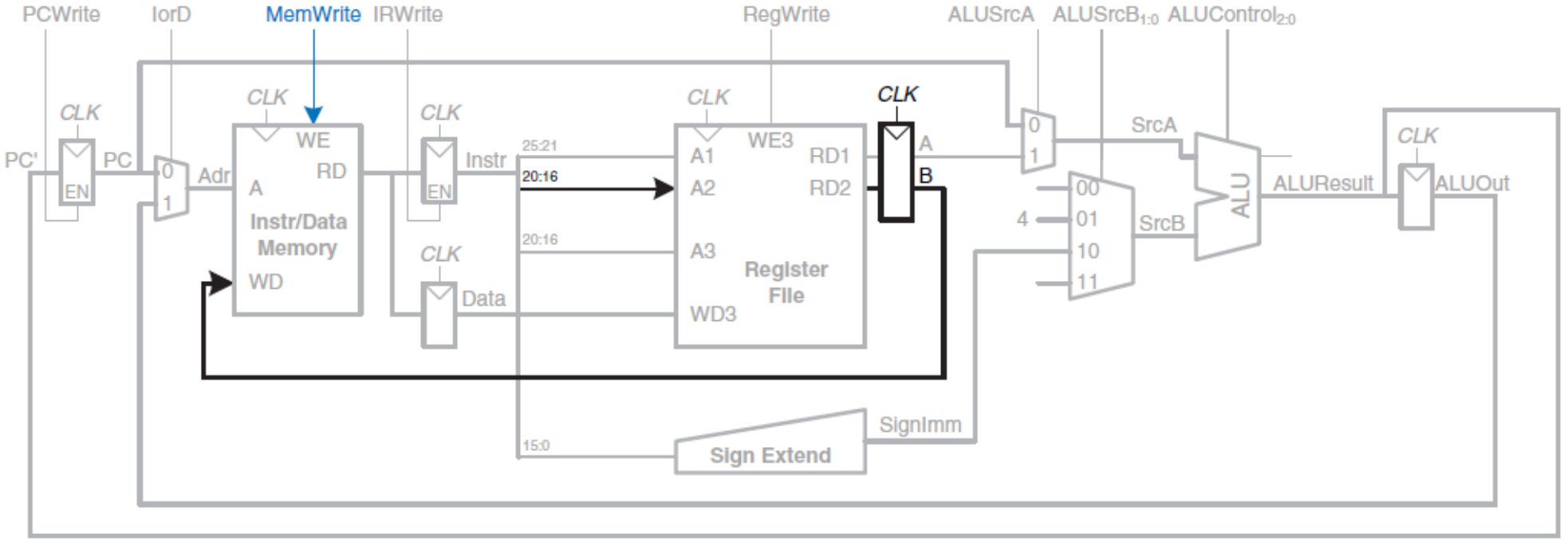
4. Multi-Cycle Processor

I-type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

To handle **sw** instruction:

- Memory address is calculated using the existing hardware in the datapath.
- A second register specifing the rt, Instr [20:16], should be read from the register file to be stored in a nonarchitectural register, B.
- The RD2 is sent to the write data port (WD) of the data memory when MemWrite = 1.



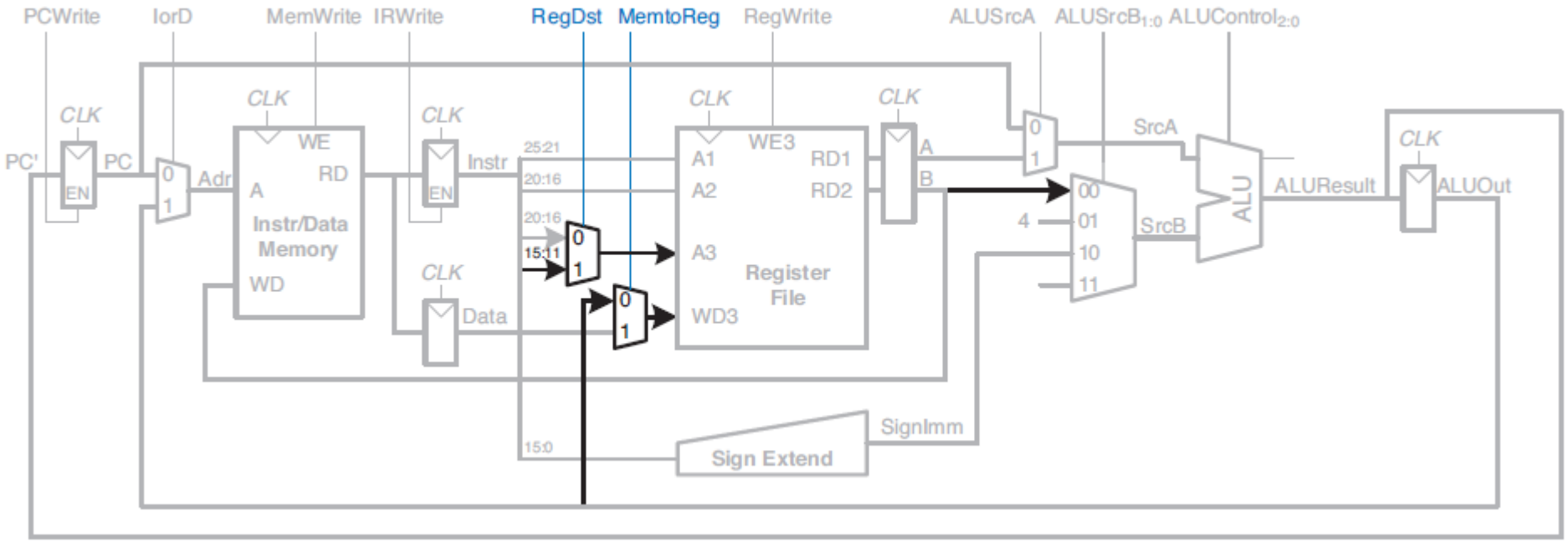
4. Multi-Cycle Processor

R-type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

To handle **R-type** instruction:

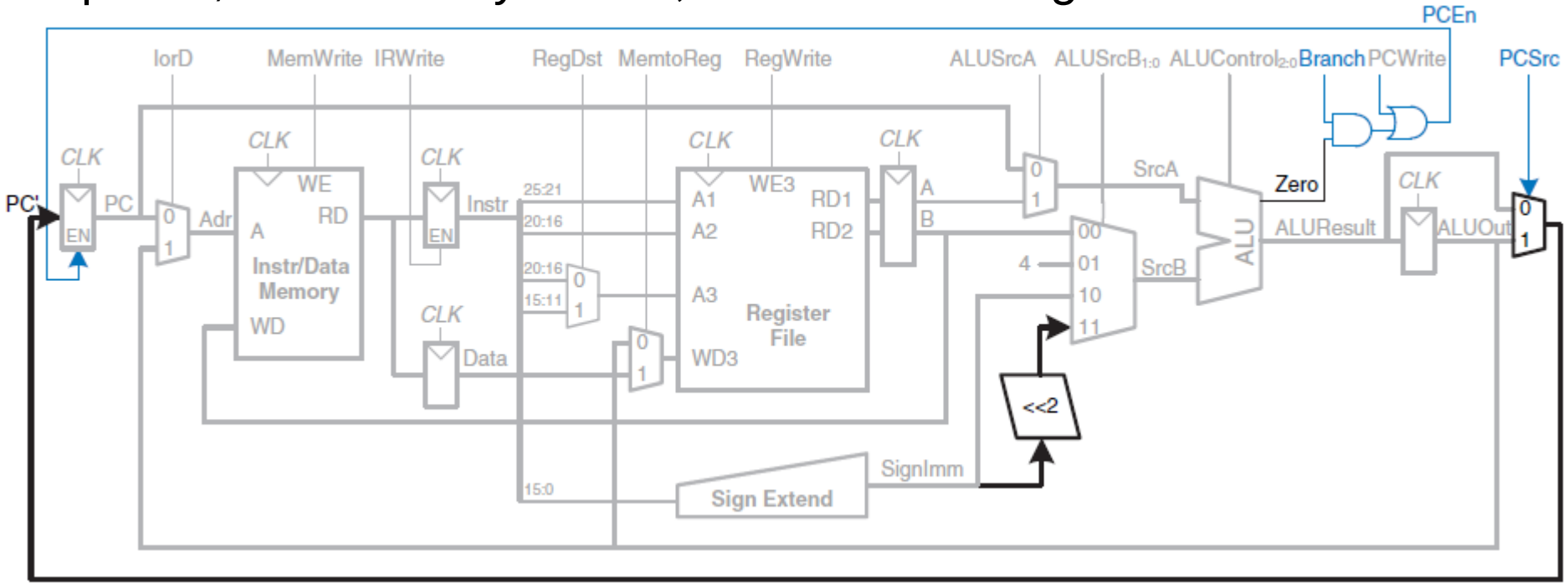
- ALUSrcB is used to choose register B as the second source register for the ALU.
- The MemtoReg mux is added to select whether WD3 comes from ALUOut (for R-type instructions) or from Data (for lw).
- The RegDst mux is added to select whether the destination register is specified in the rt (for lw/sw) or rd field (for R-type instructions) .



4. Multi-Cycle Processor

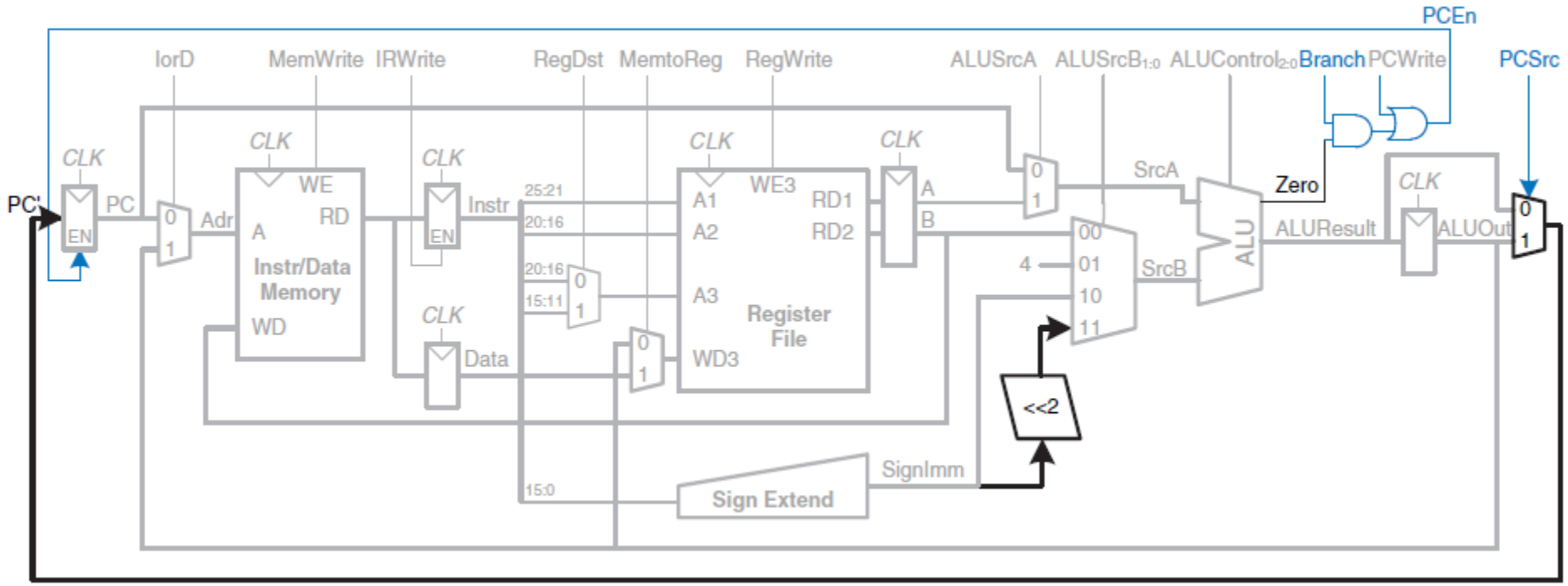
To handle the **beq** instruction:

- The next value of the PC is computed if the branch is taken.
- ALU can be reused to compute PC + SignImm × 4, which ALUSrcB multiplexer chooses and adds it to the PC.
- A new multiplexer, controlled by PCSrc, chooses what signal should be sent to PC′.



4. Multi-Cycle Processor

- A new control signal, Branch, indicates that the beq instruction is being executed.
- The program counter should be written if PC write enable, PCEn, is TRUE which is valid when PCWrite is asserted or when both Branch and Zero are asserted (branch is taken).



4. Multi-Cycle Processor

- The main difference is that the instruction is executed in several steps.
- Nonarchitectural registers are inserted to hold the results of each step.
- In this way, the ALU can be reused several times, saving the cost of extra adders.
- Similarly, the instructions and data can be stored in one shared memory.
- In the next section, we develop an FSM controller to deliver the appropriate sequence of control signals to the datapath on each step of each instruction.

4. Multi-Cycle Processor

4.1 Multi-Cycle Datapath

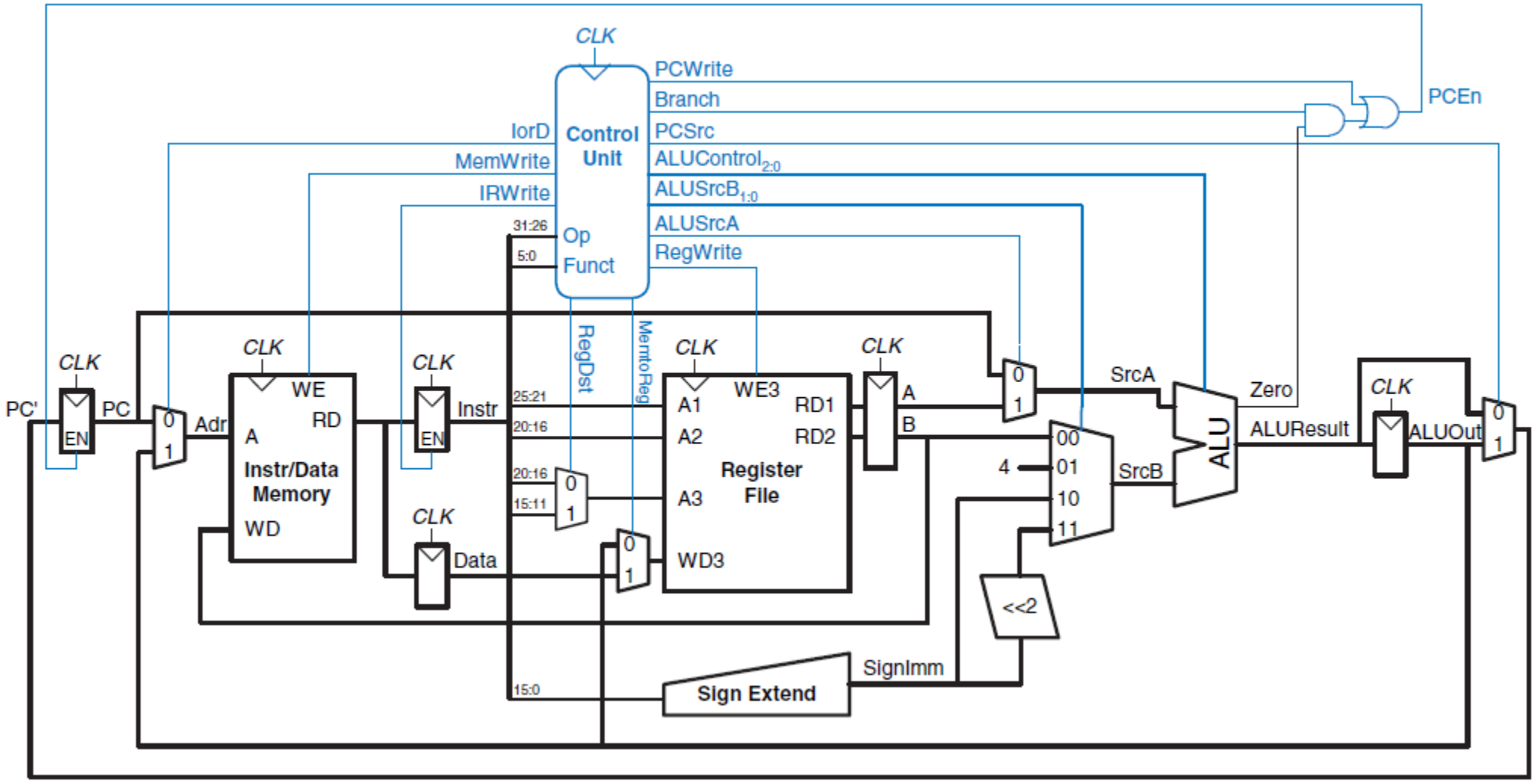
4.2 Multi-Cycle Control

3.3 More Instructions

3.4 Performance Analysis

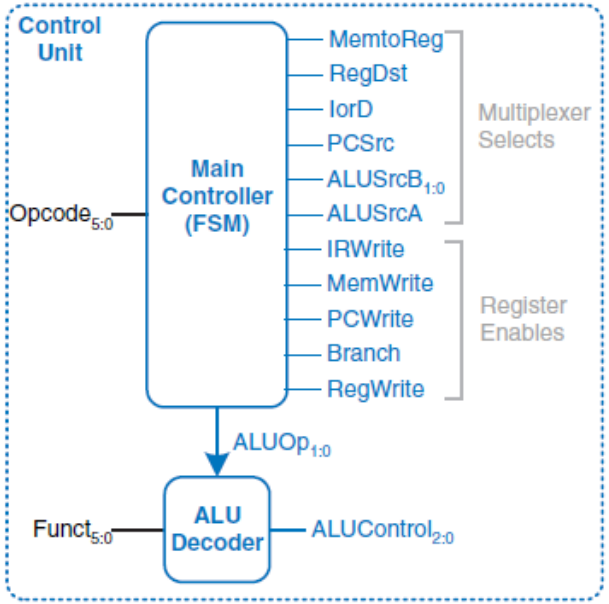
4. Multi-Cycle Processor

- The control unit computes the control signals based on the opcode, Instr [31:26], and funct fields, Instr [5:0].

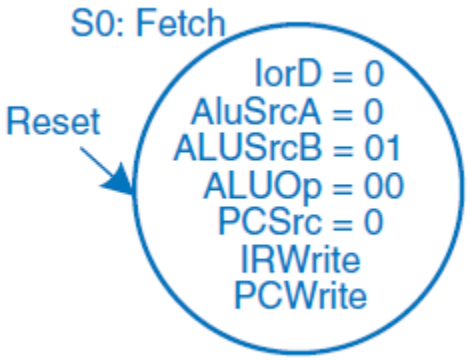


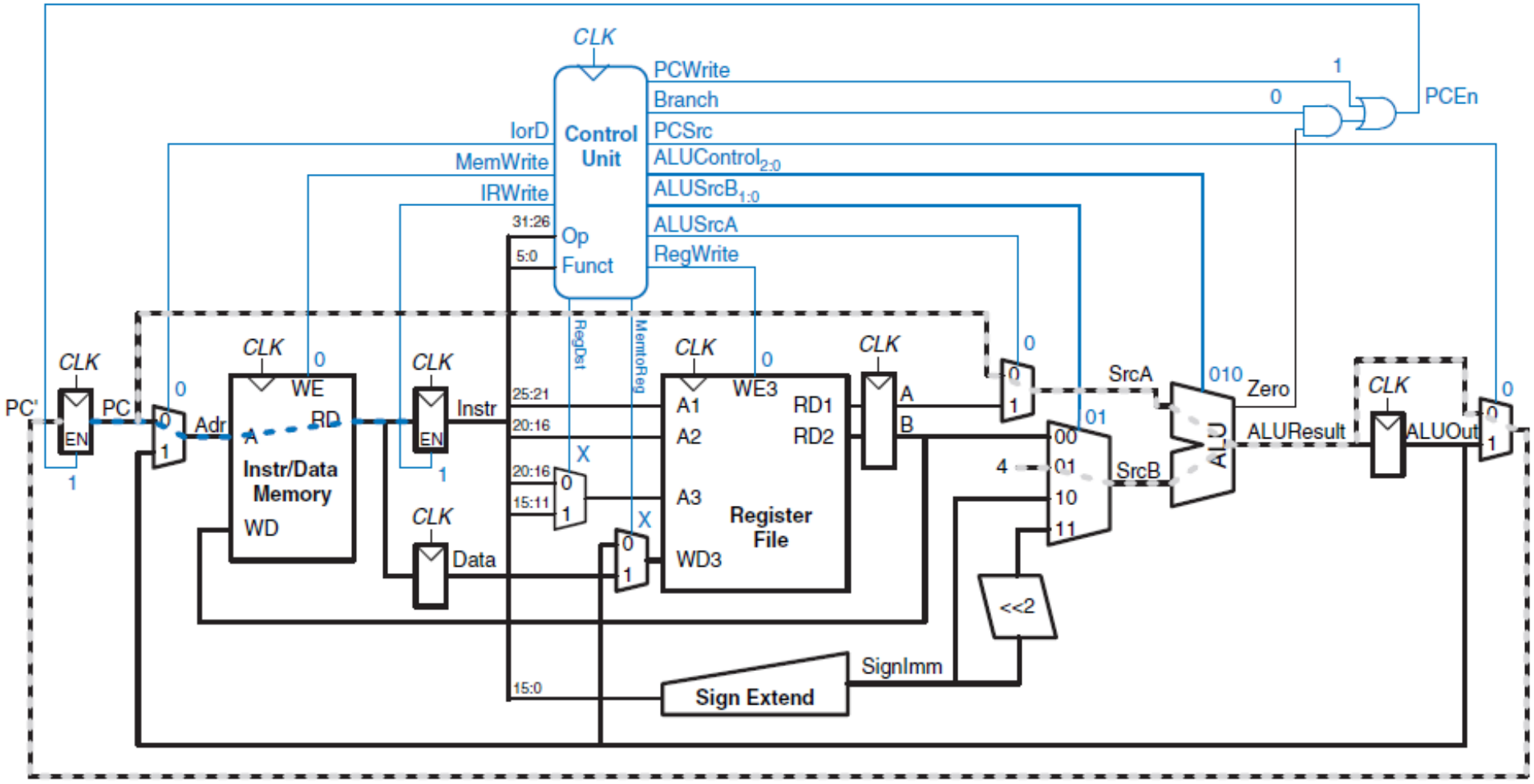
4. Multi-Cycle Processor

- The control unit is partitioned into a main controller and an ALU decoder.
- The ALU decoder is unchanged and follows the same truth table.
- The main controller is an FSM that applies the proper control signals on the proper cycles.
- The sequence of control signals depends on the instruction being executed.
- The main controller for the datapath produces:
  - The Mux select signals:
    MemtoReg, RegDst, IorD, PCSrc, ALUSrcB, and ALUSrcA.
  - The Reg/Mem enable signals:
    IRWrite, MemWrite, PCWrite, Branch, and RegWrite.



4. Multi-Cycle Processor



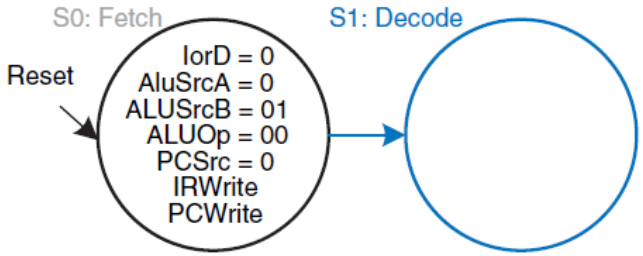


4. Multi-Cycle Processor

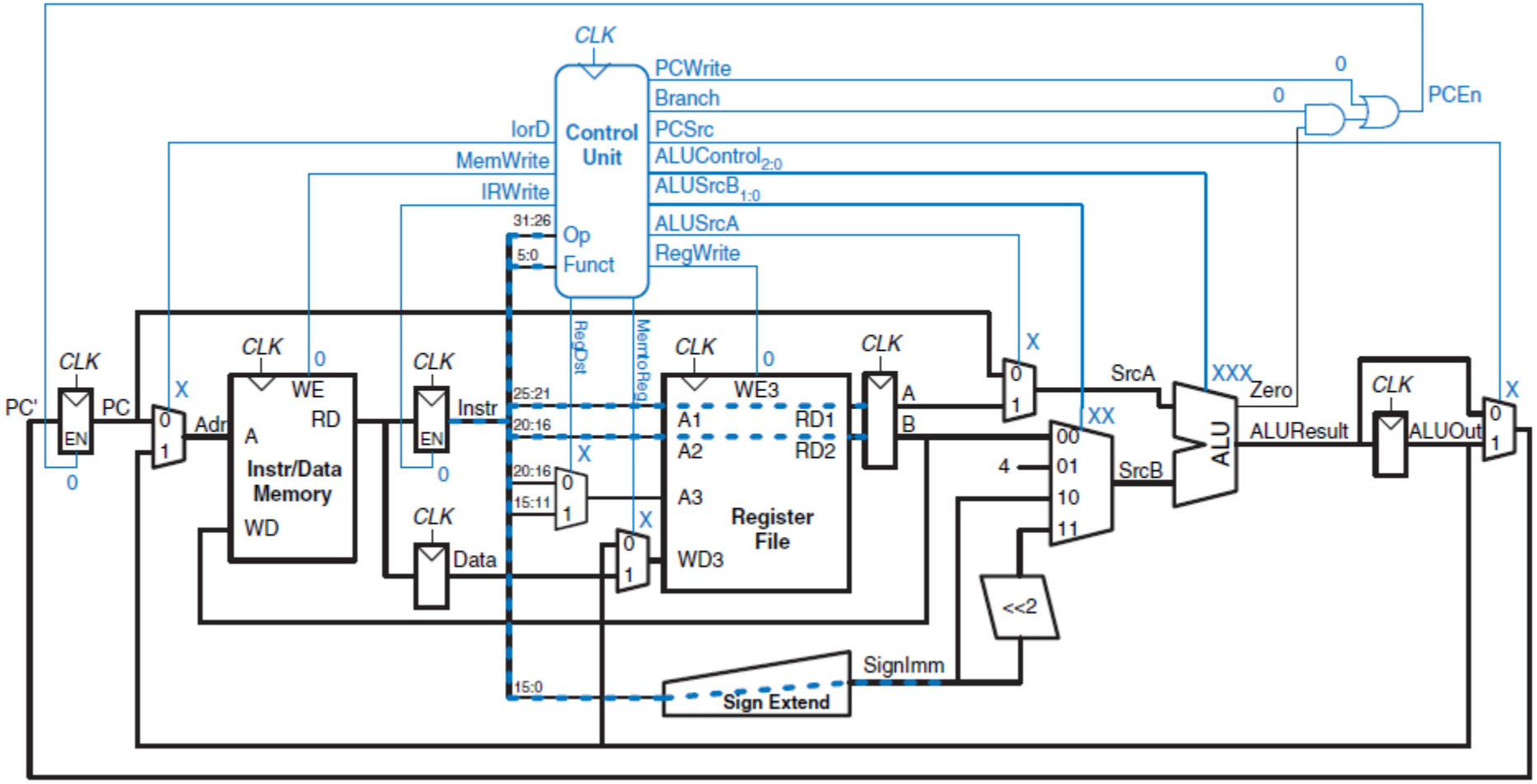
- The first step is to **fetch** the instruction from memory at the PC address.
- The FSM enters this state on **reset**.
- To read memory, IorD= 0, so the address is taken from the PC.
- IRWrite is asserted to write the instruction into the instruction register. The PC should be incremented by 4 to point to the next instruction.



- Because the ALU is not being used, the processor can use it to compute PC + 4 while it fetches the instruction.
- ALUSrcA = 0, so SrcA comes from the PC. ALUSrcB = 01, so SrcB is the constant 4.
- ALUOp = 00, so the ALU decoder produces ALUControl = 010 to make the ALU add.
- To update the PC with this new value, PCSrc = 0, and PCWrite is asserted.

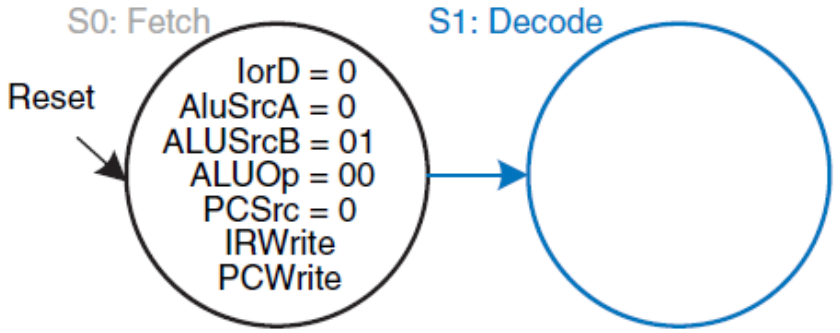


4. Multi-Cycle Processor

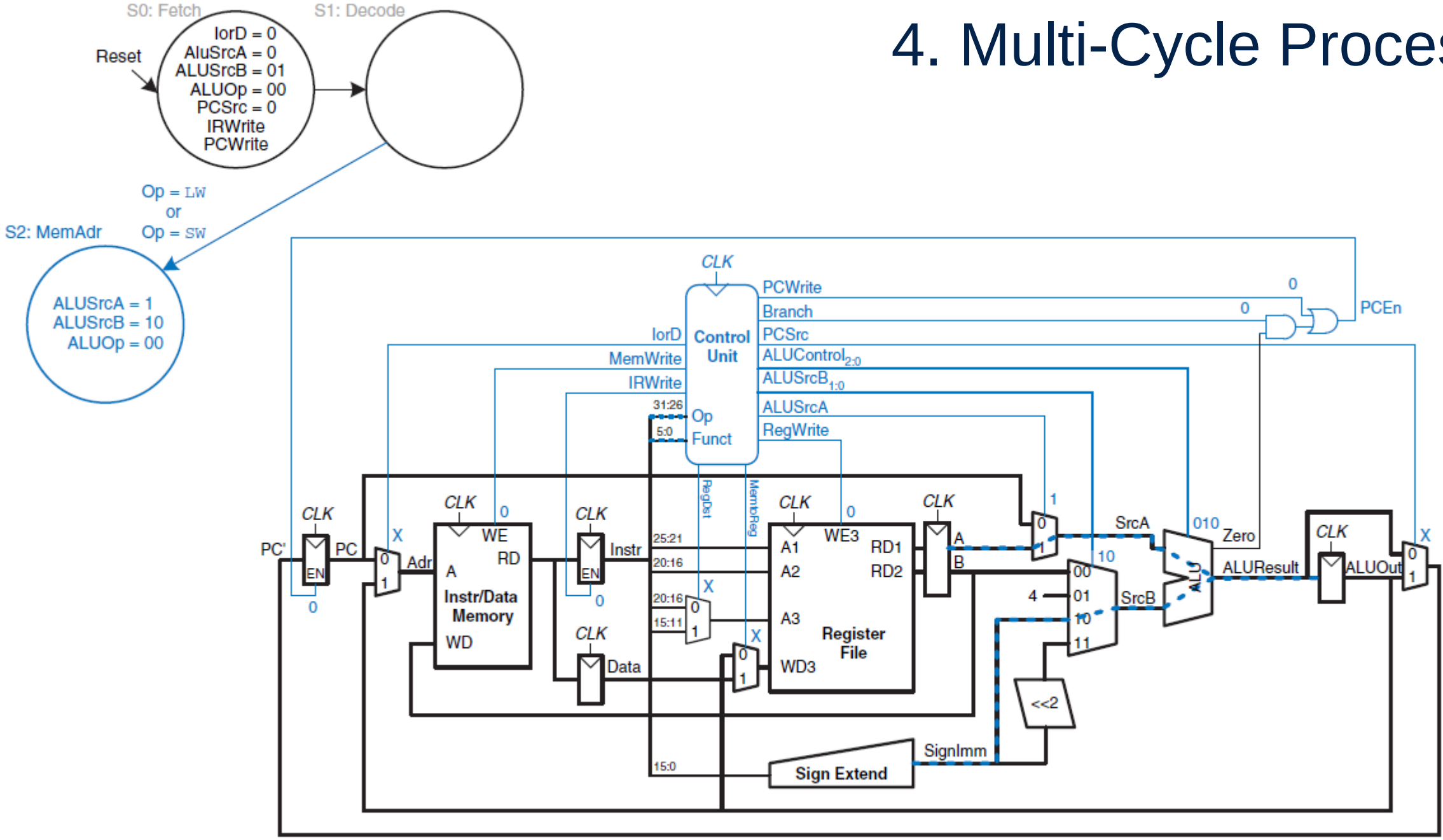


4. Multi-Cycle Processor

- The next step is to read the register file and decode the instruction.
- The register file always reads the two sources (rs and rt fields) of the instruction.
- Decoding involves examining the opcode of the instruction to determine what to do next.
- No control signals are necessary to decode the instruction, but the FSM must wait 1 cycle for the reading and decoding to complete.

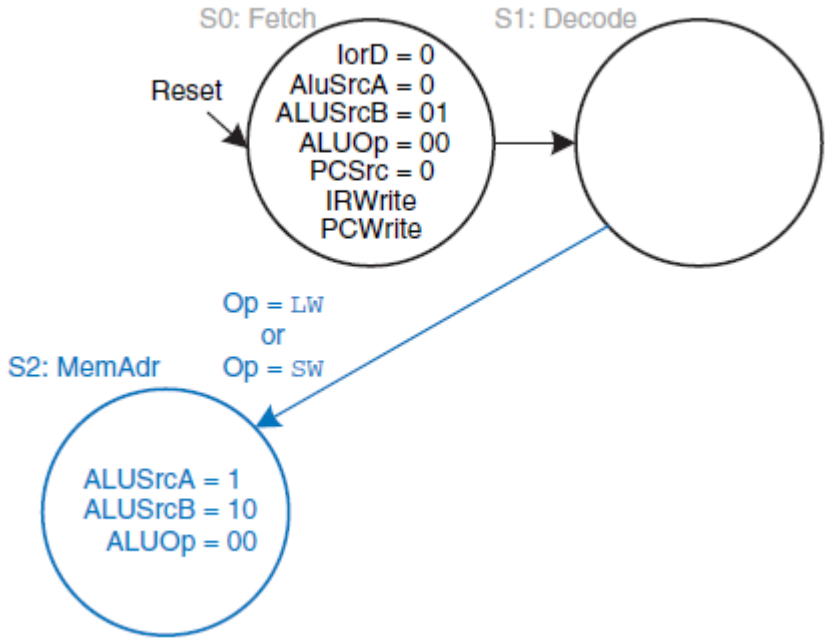


4. Multi-Cycle Processor



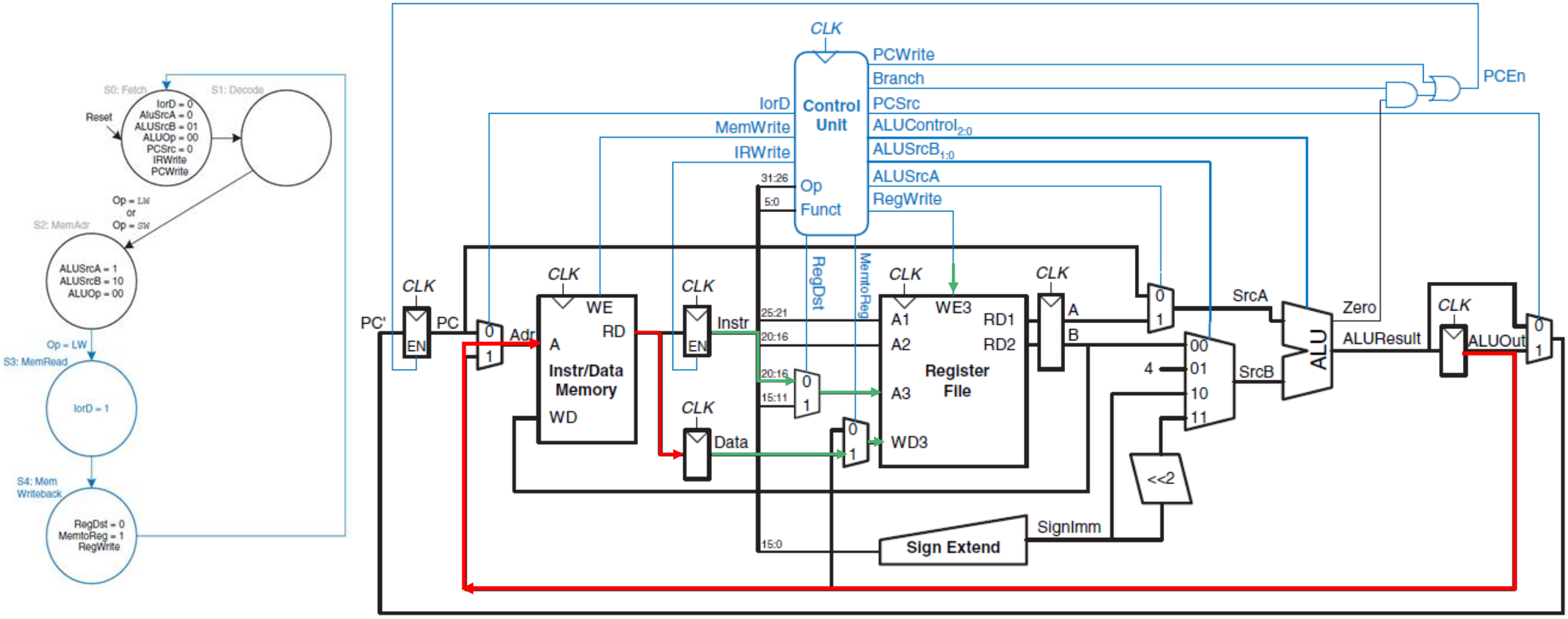
4. Multi-Cycle Processor

- The FSM proceeds to one of several possible states, depending on the opcode.
- If the instruction is a memory load or store (**lw** or **sw**), the multicycle processor computes the address by adding the base address to the sign-extended immediate.
- This requires ALUSrcA = 1 to select register A.
- ALUSrcB = 10 to select SignImm.
- ALUOp = 00, so the ALU adds.
- The effective address is stored in the ALUOut register for use on the next step.



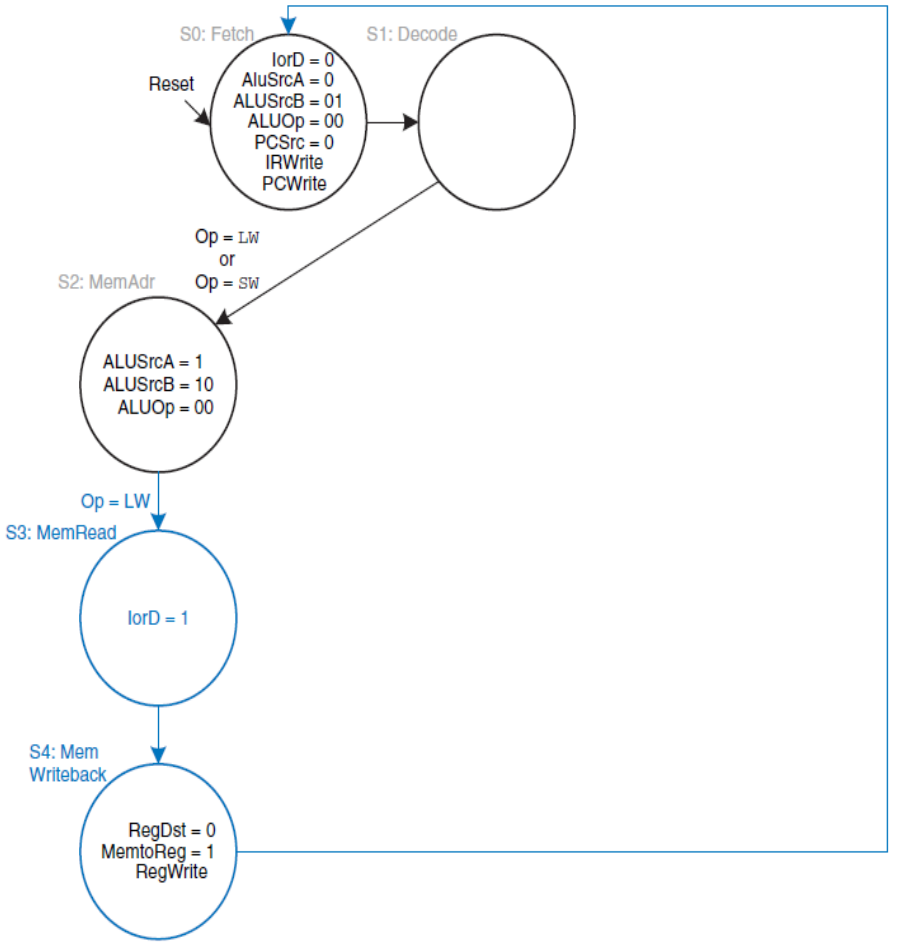
4. Multi-Cycle Processor

- In case of **lw** instruction:



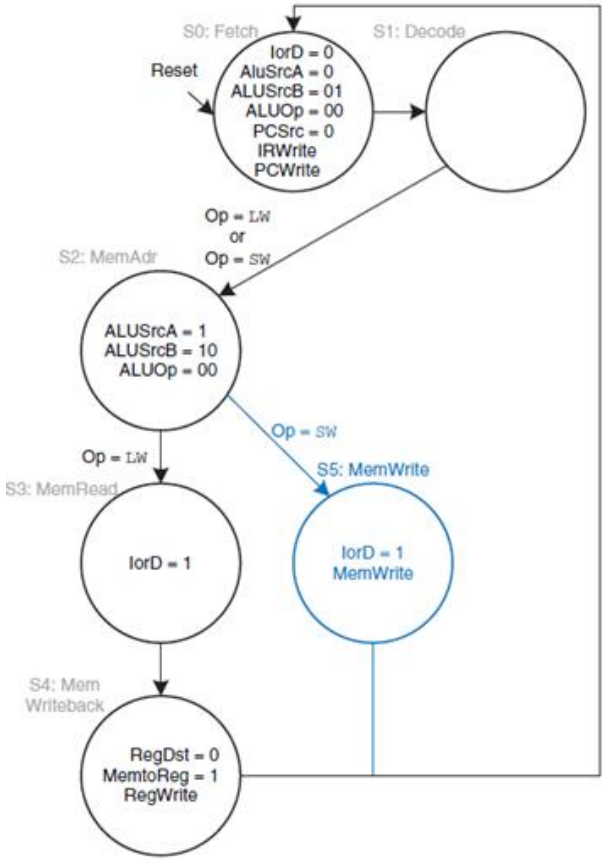
4. Multi-Cycle Processor

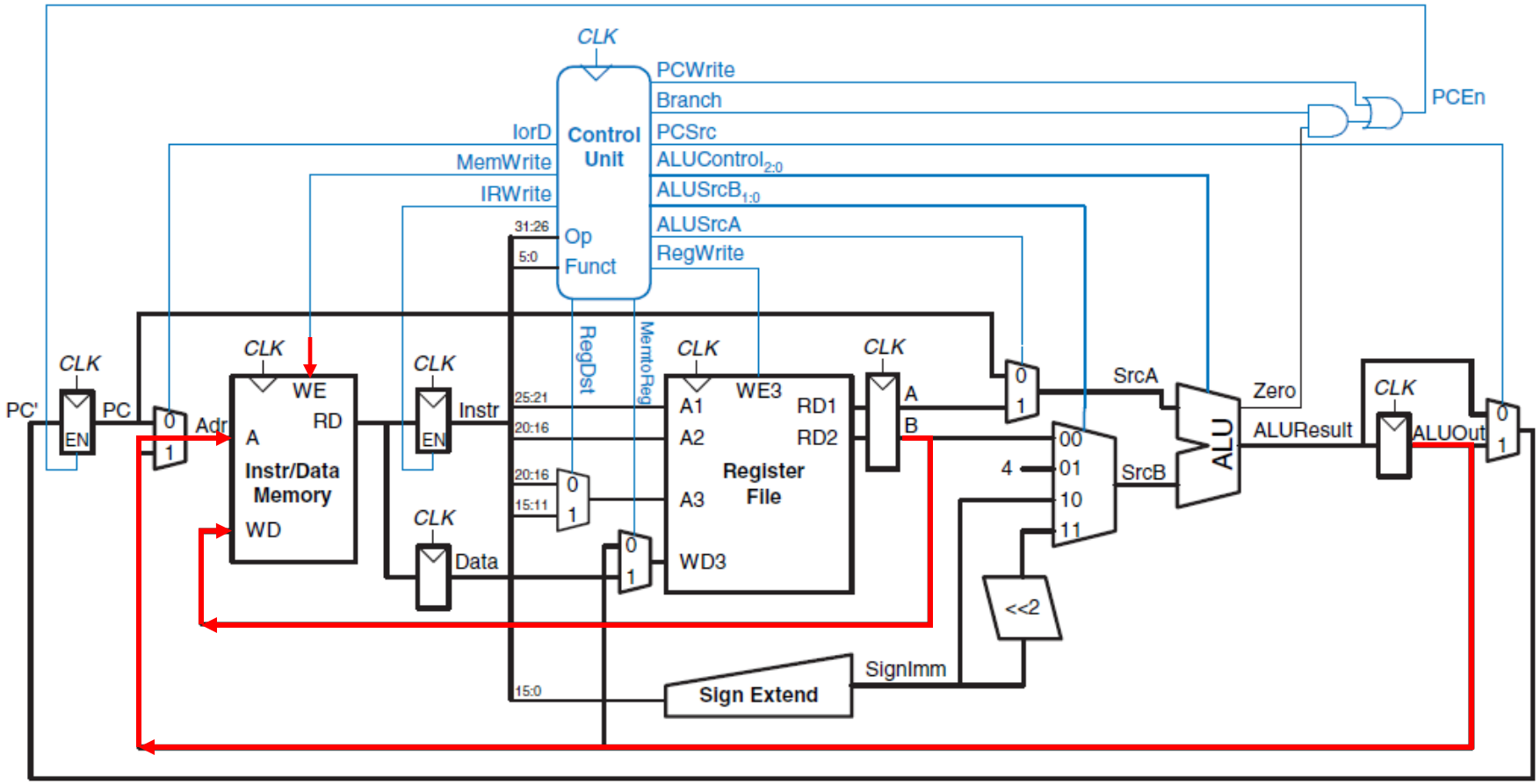
- If the instruction is **lw**, the multicycle processor must read data from memory and write it to the reg file.
- To read from memory, IorD = 1 to select the memory address that was just computed and saved in ALUOut.
- This address in memory is read and saved in the Data register during step S3.
- On the next step, S4, Data is written to the register file.
- MemtoReg = 1 to select Data, and RegDst = 0 to pull the destination register from rt of the instruction.
- RegWrite is asserted to perform the write.
- Finally, FSM returns to S0 to fetch the next instruction.



4. Multi-Cycle Processor

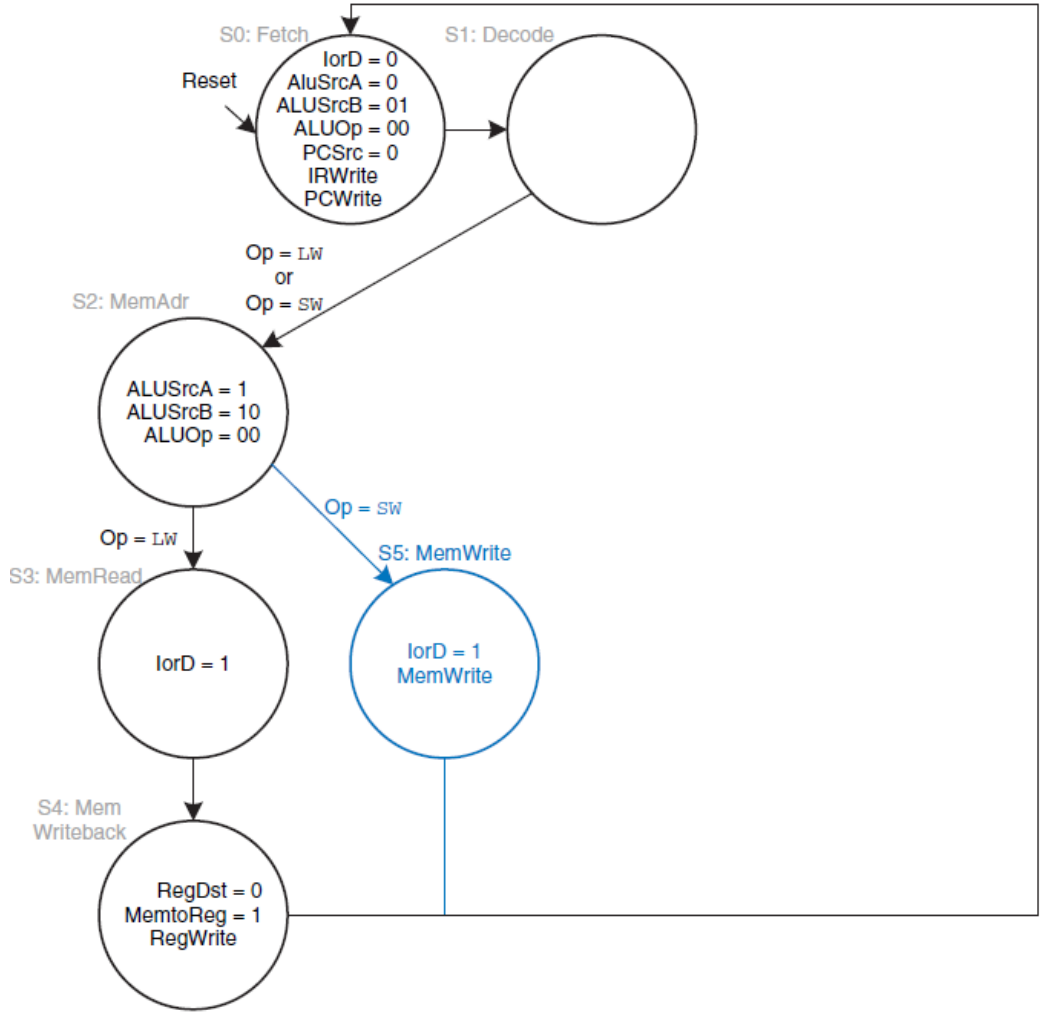
- In case of **sw** instruction.





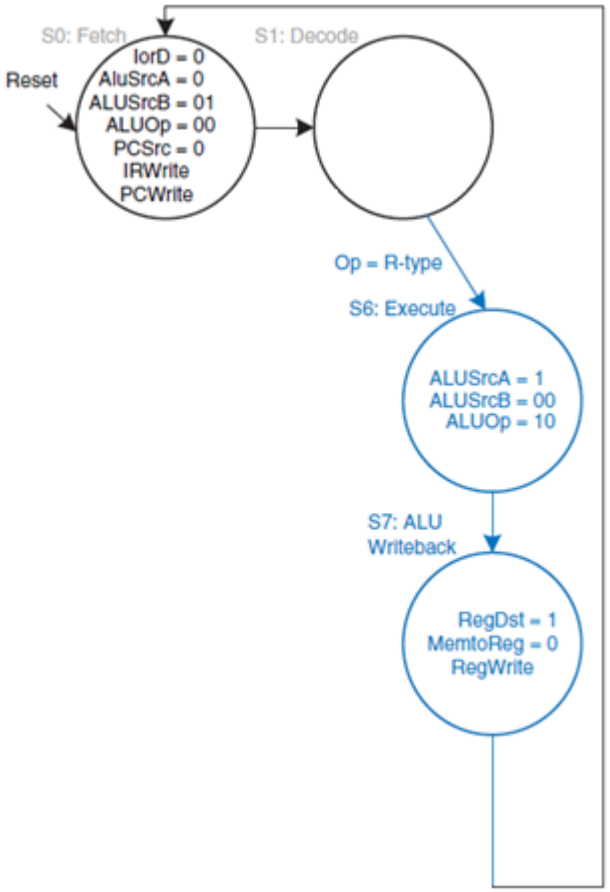
4. Multi-Cycle Processor

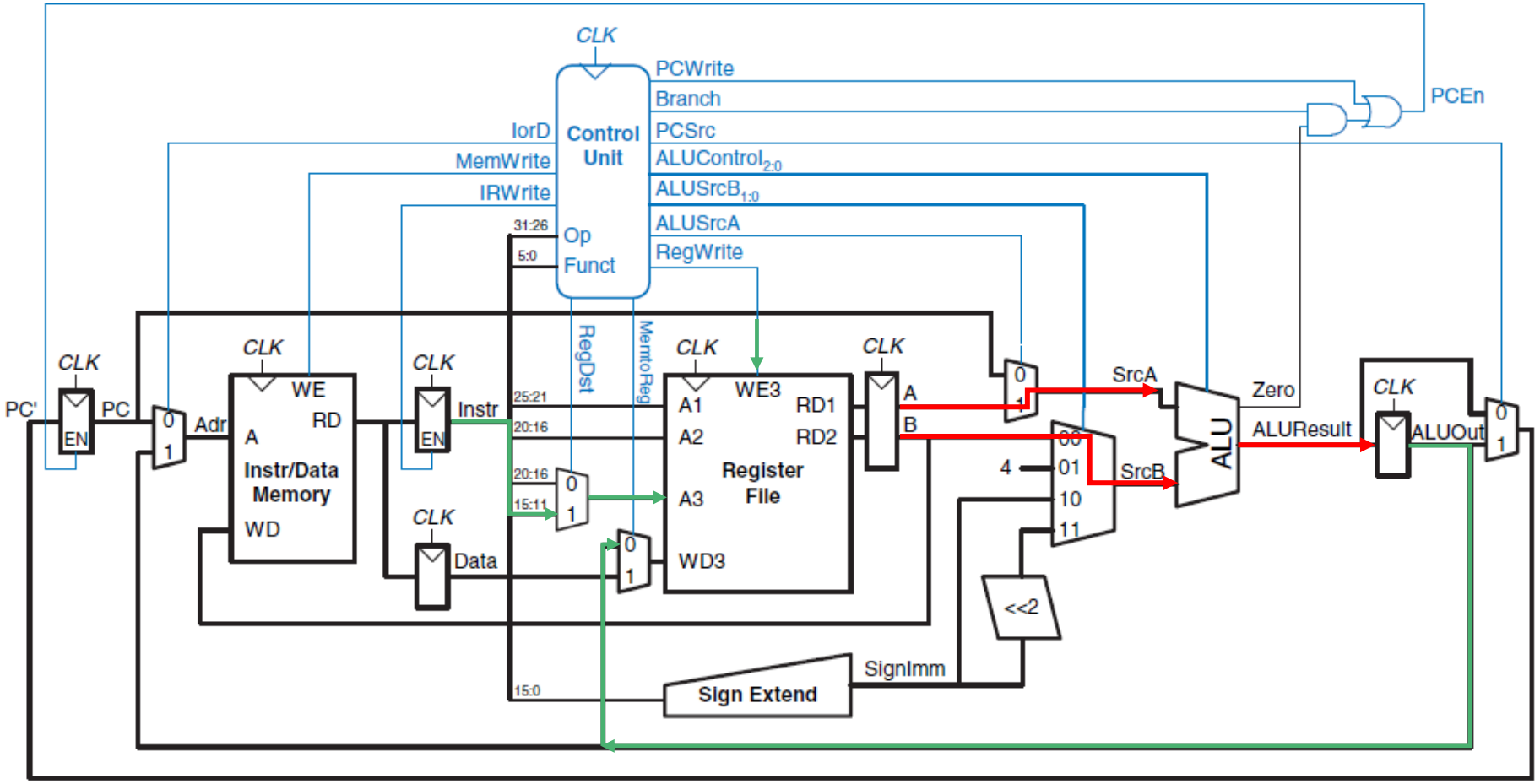
- From state S2, if the instruction is **sw**, the data read from the second port of the register file is simply written to memory.
- In state S5, IorD = 1 to select the address computed in S2 and saved in ALUOut.
- MemWrite is asserted to write the memory.
- Again, the FSM returns to S0 to fetch the next instruction.



4. Multi-Cycle Processor

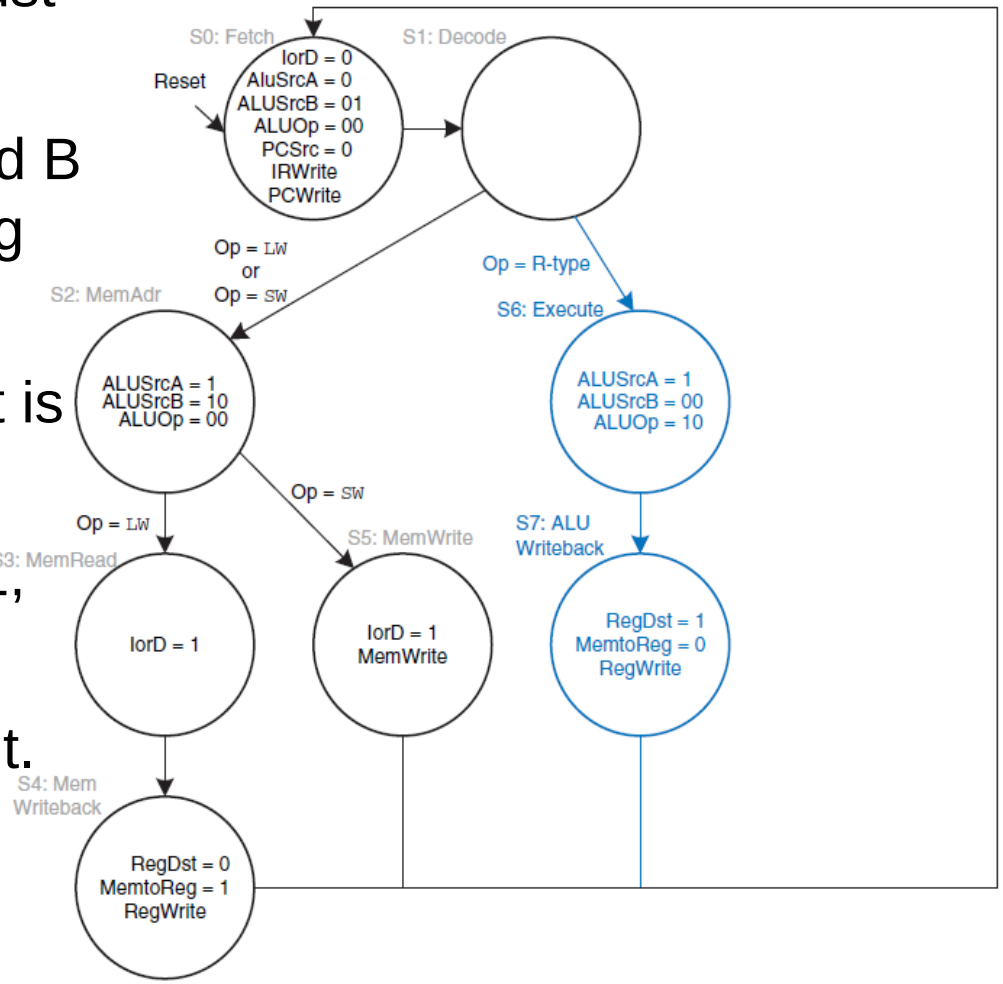
- In case of R-type instructions.





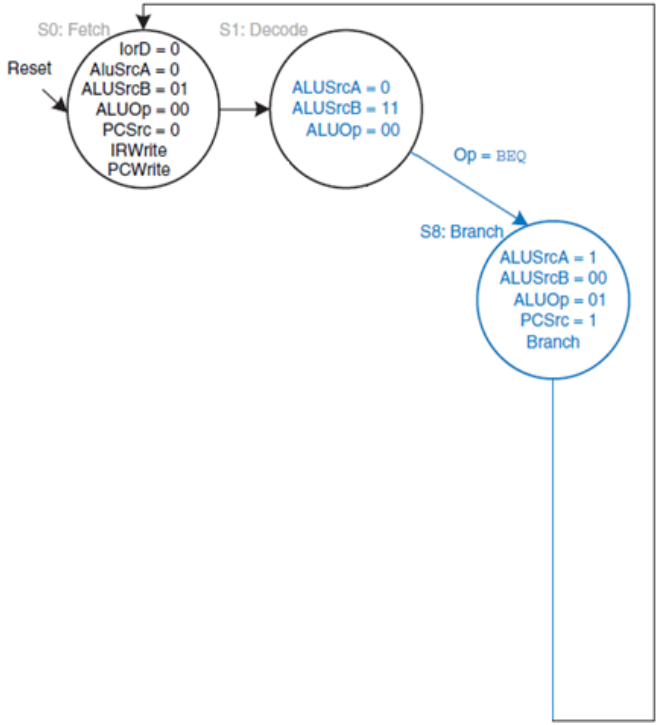
4. Multi-Cycle Processor

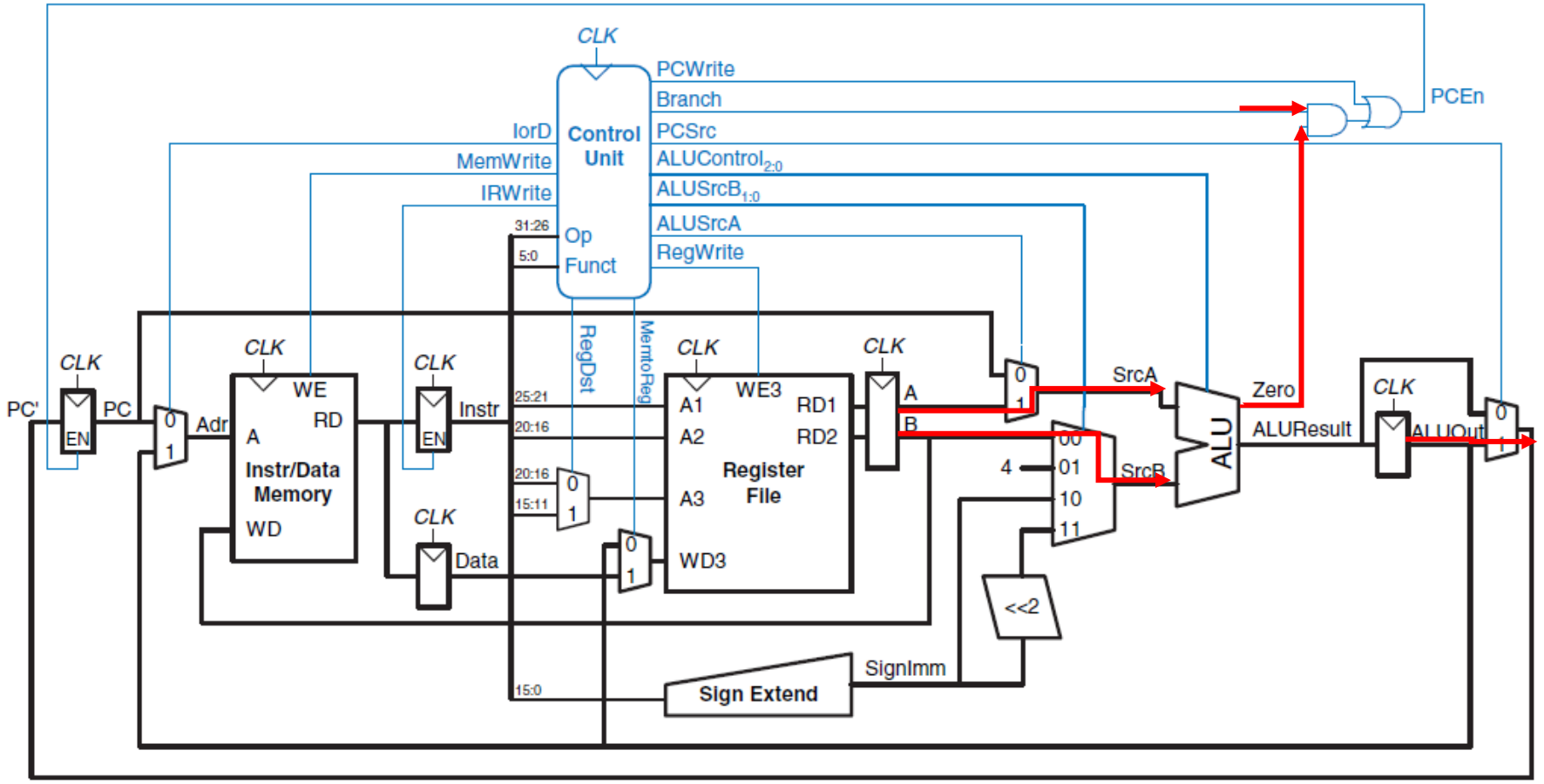
- For the R-type instruction, the multicycle processor must calculate the result using the ALU and write to the RF.
- In S6, the instruction is executed by selecting the A and B registers (ALUSrcA = 1, ALUSrcB = 00) and performing the ALU operation (funct field in the instruction).
- ALUOp = 10 for all R-type instructions. The ALUResult is stored in ALUOut.
- In S7, ALUOut is written to the register file, RegDst = 1, to consider the dest register (rd field in instruction).
- MemtoReg = 0 as the write data, WD3, is from ALUOut.
- RegWrite is asserted to write the register file.



4. Multi-Cycle Processor

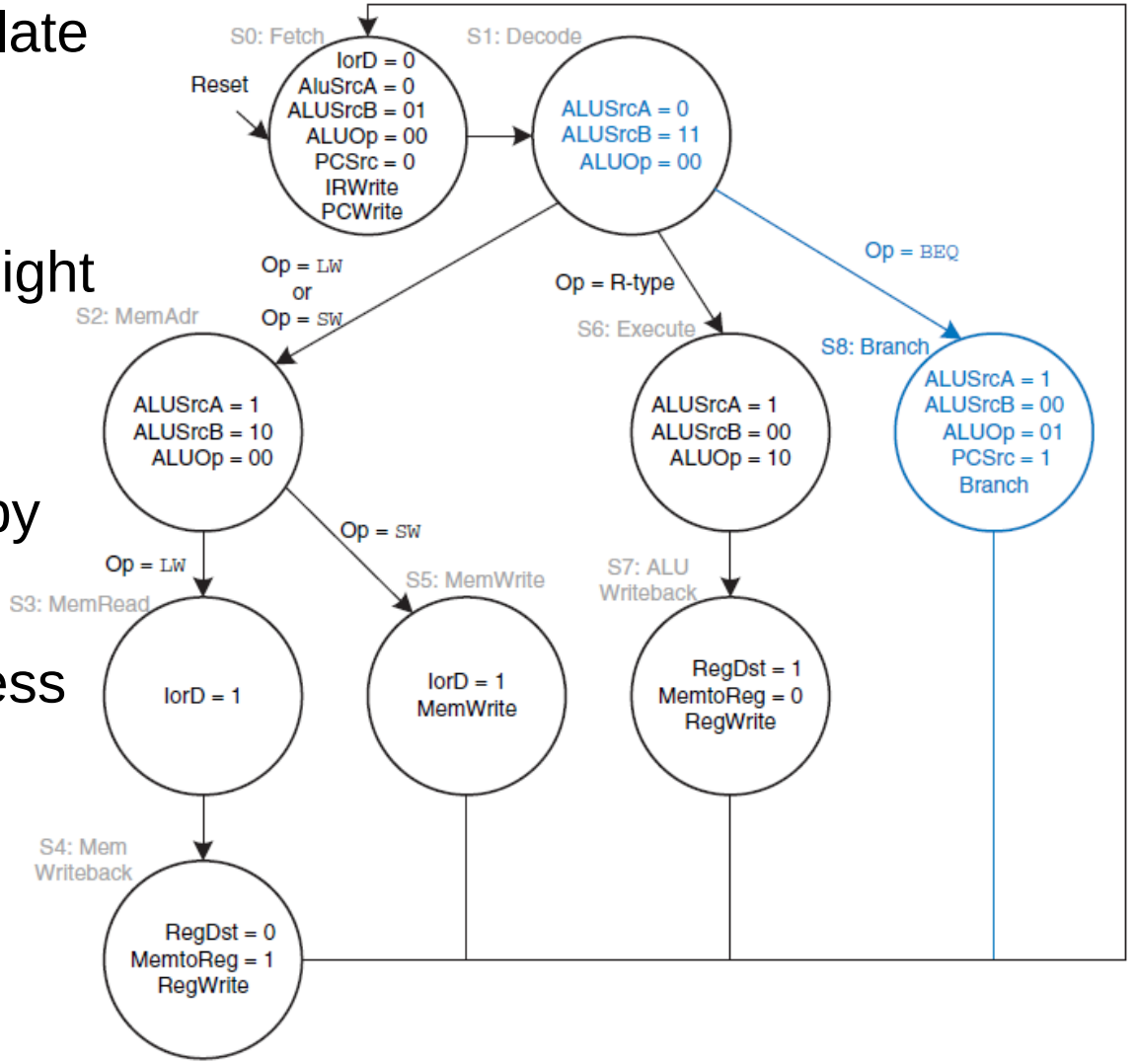
- In case of **beq** instruction:
- State Reduction: Use Decode stage to add the offset





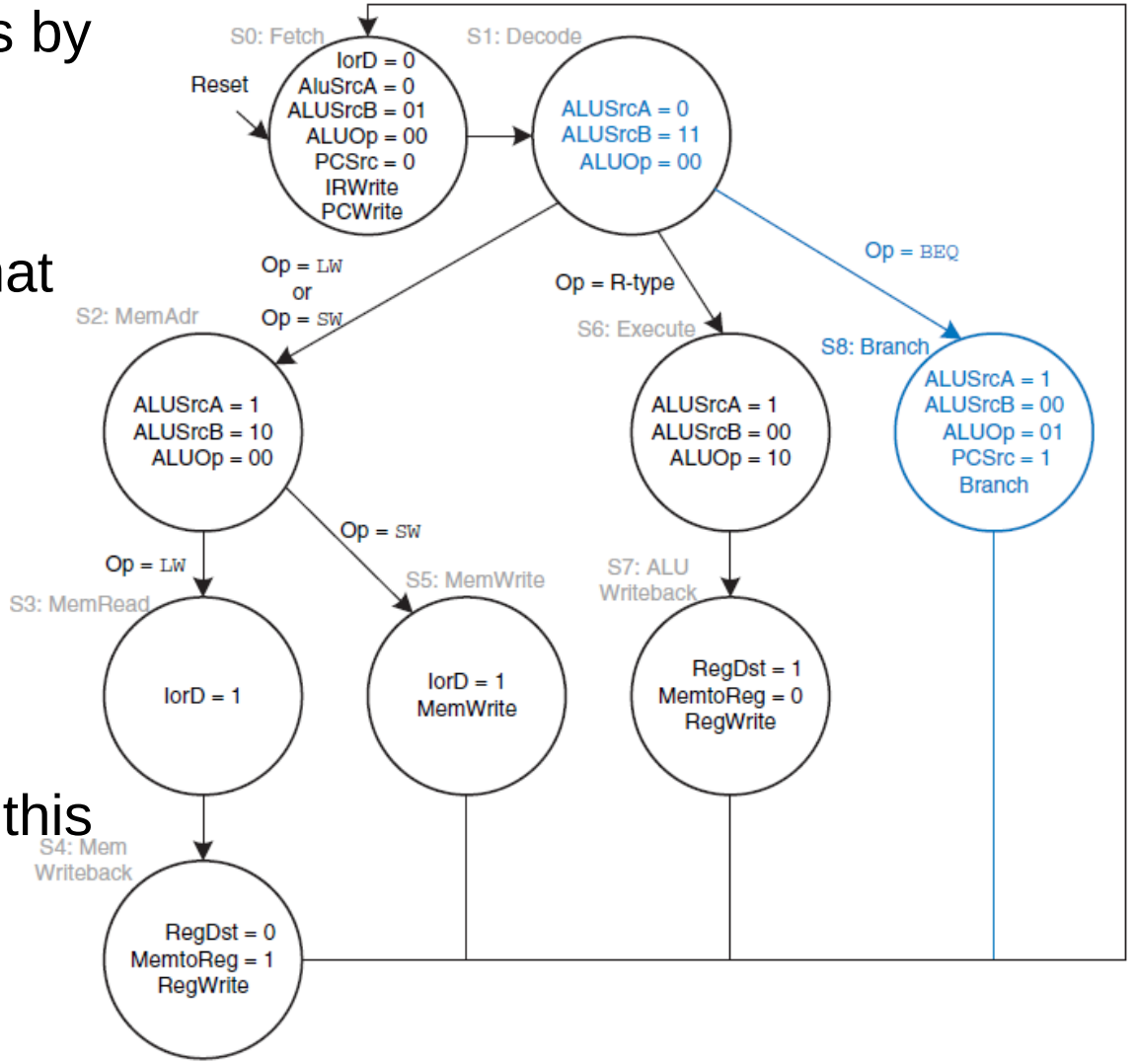
4. Multi-Cycle Processor

- For a **beq** instruction, the processor must calculate the dest address and compare the two source registers to determine if the branch is taken.
- This requires two uses of the ALU and hence might seem to demand two new states.
- The processor might use the ALU during S1 to compute the dest address, as it was not used, by adding the (PC + 4) to (SignImm×4).
- If the instruction is not beq, the computed address will not be used in subsequent cycles, but its computation was harmless.



4. Multi-Cycle Processor

- In S8, the processor compares the two registers by subtracting them and checking to determine whether the result is 0.
- If it is, the processor branches to the address that was just computed.
- ALUSrcA = 1 to select register A.
- ALUSrcB = 00 to select register B.
- ALUOp = 01 to subtract.
- PCSrc = 1 to take the destination address from ALUOut, and Branch = 1 to update the PC with this address if the ALU result is 0.



4. Multi-Cycle Processor

4.1 Multi-Cycle Datapath

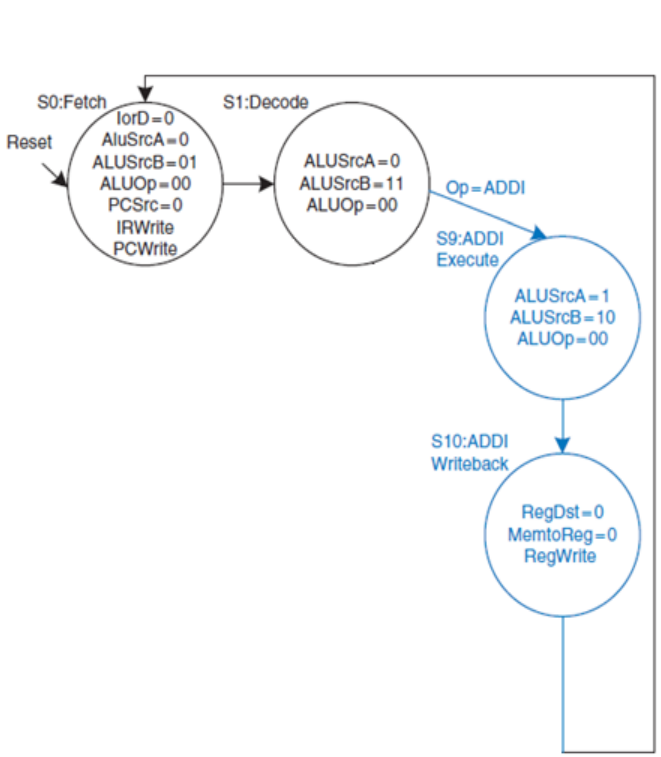
4.2 Multi-Cycle Control

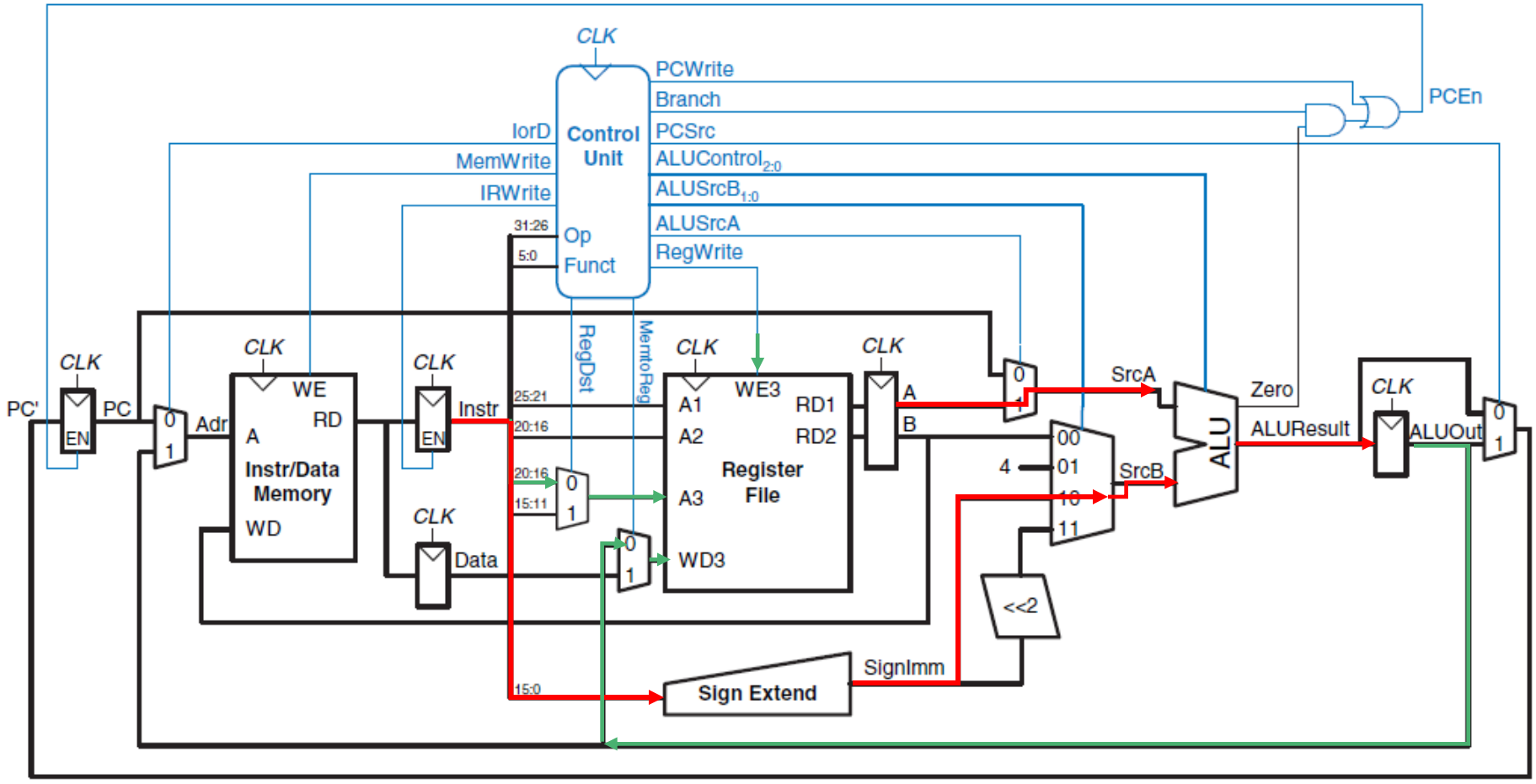
4.3 More Instructions

4.4 Performance Analysis

4. Multi-Cycle Processor

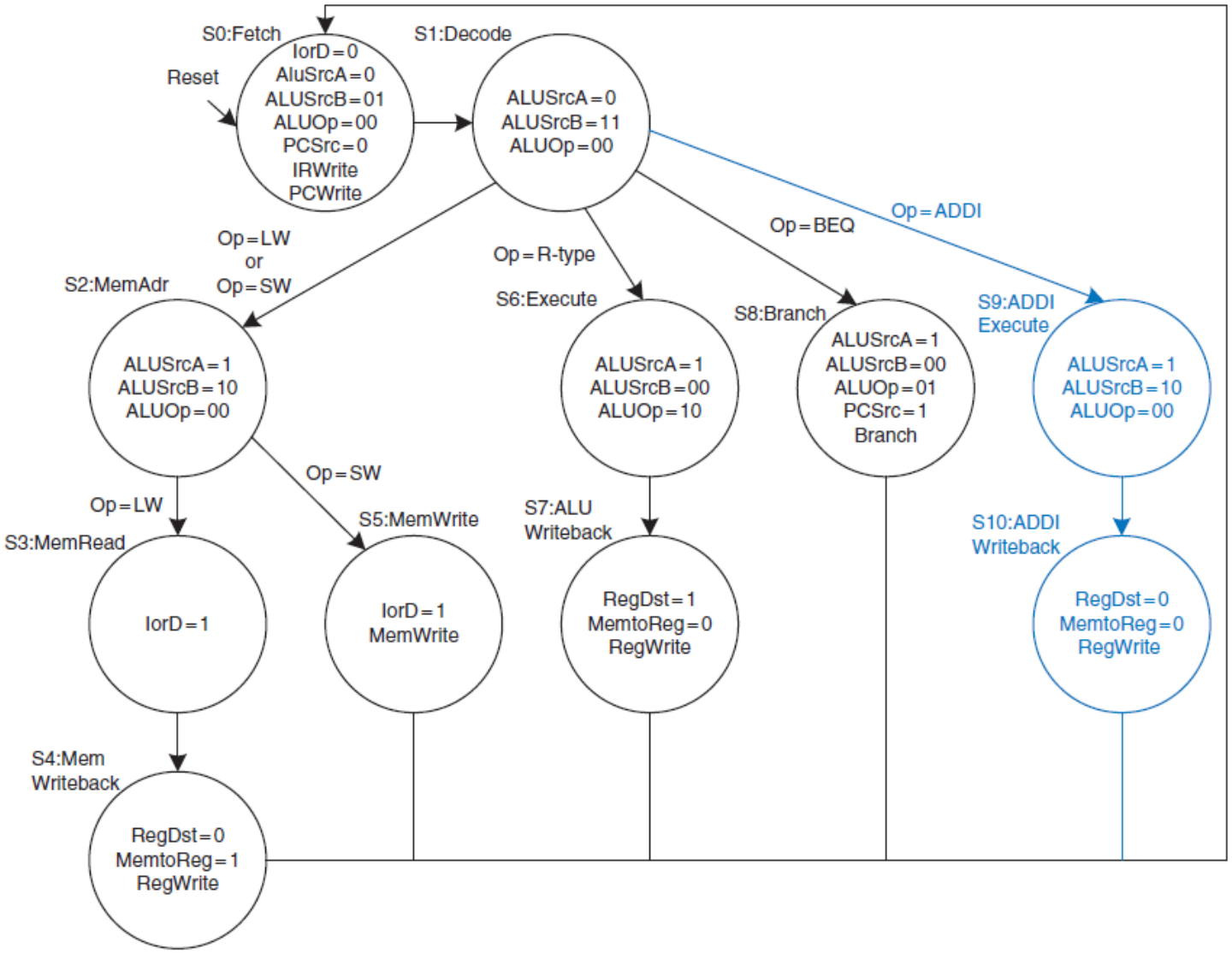
- In case of **addi** instruction:





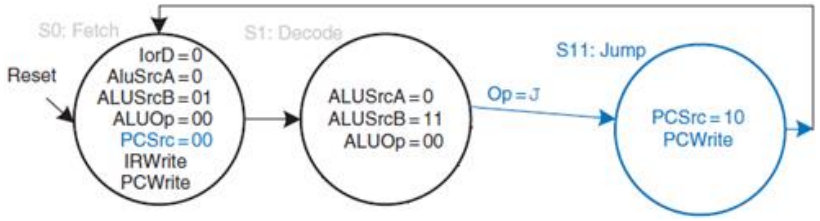
4. Multi-Cycle Processor

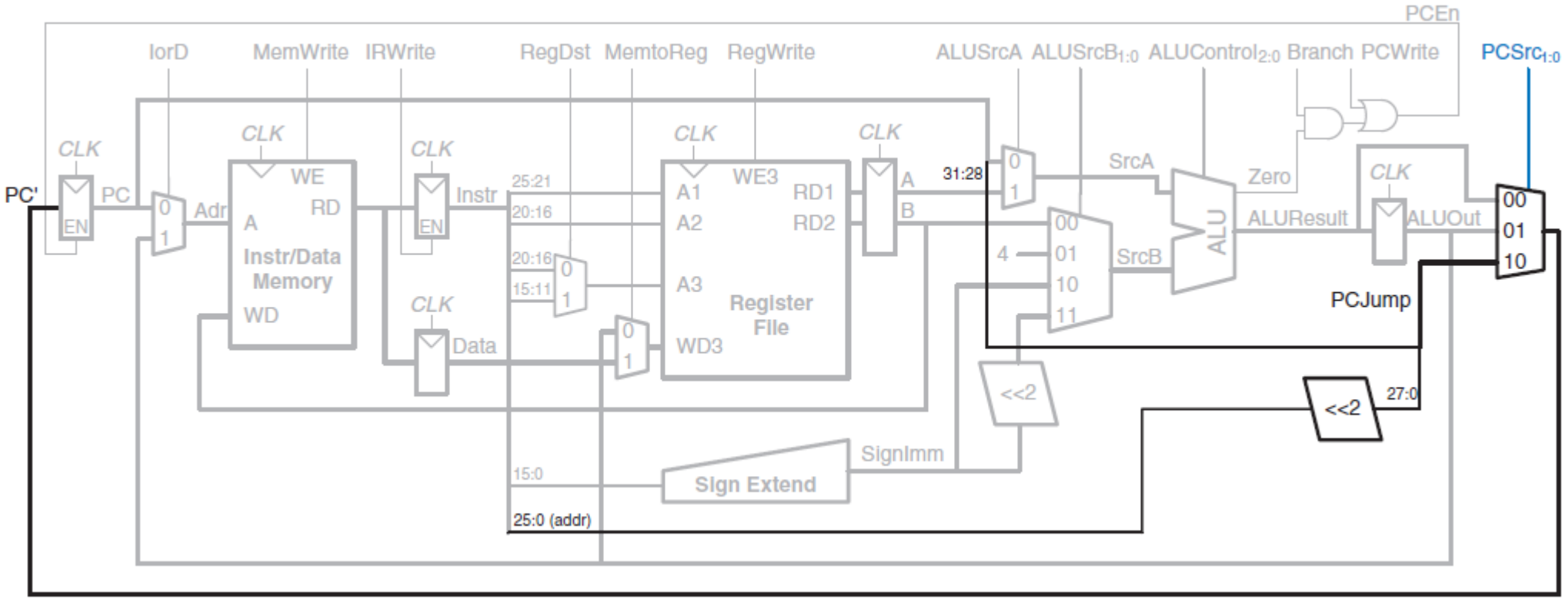
- For addi Instruction



4. Multi-Cycle Processor

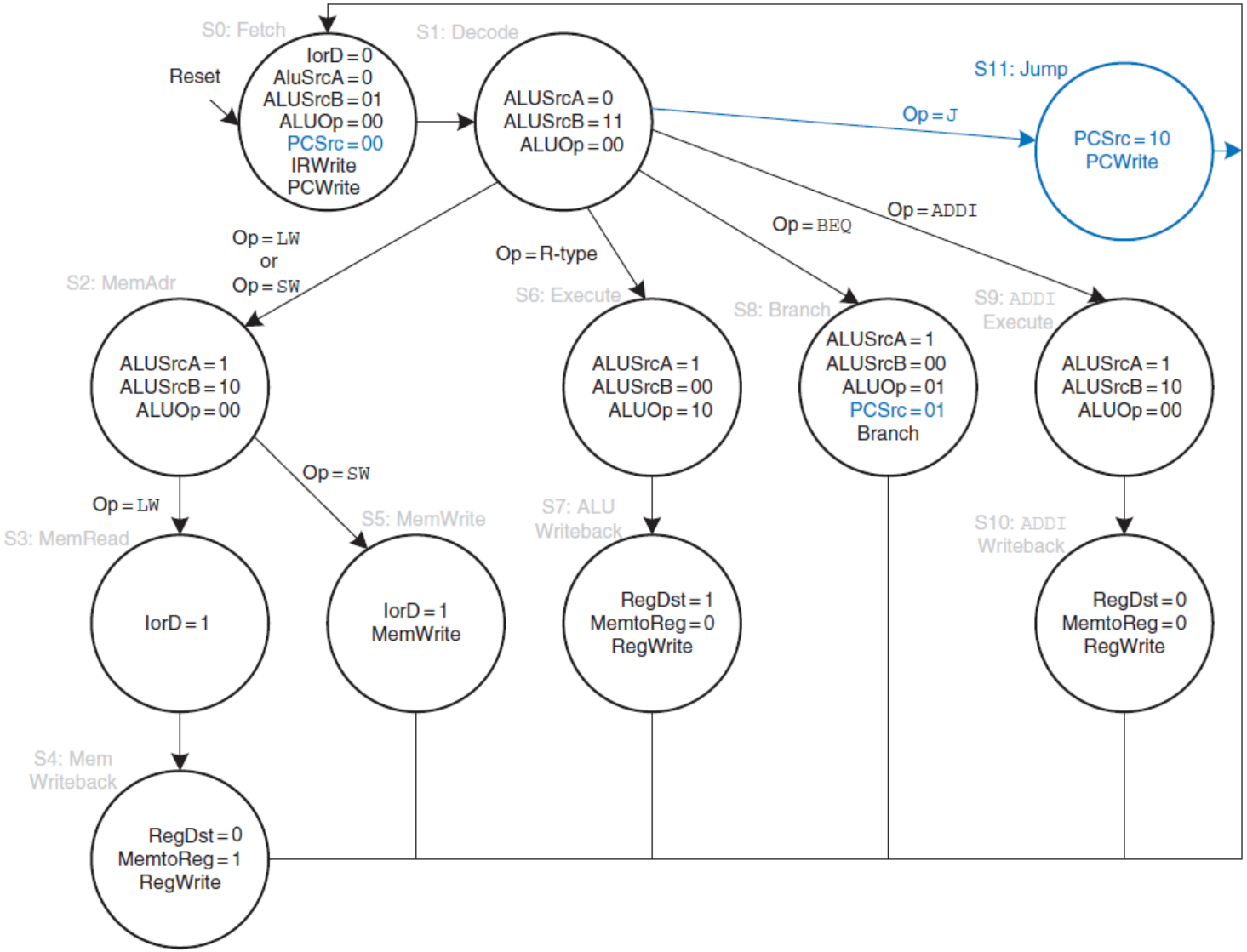
- In case of **j** instruction:





4. Multi-Cycle Processor

- For **j** instruction



4. Multi-Cycle Processor

4.1 Multi-Cycle Datapath

4.2 Multi-Cycle Control

4.3 More Instructions

4.4 Performance Analysis

4. Multi-Cycle Processor

- The execution time of an instruction depends on both the CPI and T_c.
- This processor uses many cycles depending on the instruction, but its T_c is shorter.
- The multicycle processor requires **3 cycles** for **beq** and **j** instructions, **4 cycles** for **sw**, **addi**, and **R-type** instructions, and **5 cycles** for **lw** instructions.
- Each cycle involved one ALU operation, memory access, or register file access.
  Assume that the register file is faster than the memory ($t_{mem} > t_{RF}$).
  Then, two possible critical paths would limit the cycle time.

$$T_c = t_{pcq} + t_{mux} + \max(t_{ALU} + t_{mux}, t_{mem}) + t_{setup}$$

- The numerical values of these times depend on the specific implementation technology.

Assignment #3

Assignment #3

- Implement the **full** SingleCycle MIPS Processor which supports all the mentioned instructions, documented in *Appendix B, except floating point instructions*.
- Implement Dadda Multiplier and Non-Restoring Divider, using indirect method for signed numbers.
- Apply the given tests [Test#1..6] to verify the functionality.
- [As a plus] Apply a randomized testbench, do the required constraints and the possible assertions
- Deadline: Friday, 8th Nov 2024, at 11:59 PM.
- Deliveries: Design files, testbench, and a presentation for your work.
  Send a mail to: eng.abdulkareem.abotaleb@gmail.com
  Using a title of: CompArch_assignment3_<first_name>_<last_name>